

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

Xây dựng trang web hỗ trợ viết CV có tích hợp AI

HỘI ĐỒNG: Khoa học Máy tính - 12 CLC
GVHD: ThS. Mai Đức Trung
TKHD: KS. Nguyễn Minh Tâm
—o0o—

SVTH: Nguyễn Thành Vinh - 2053592

TP. HỒ CHÍ MINH, THÁNG 08/2026

Chữ ký của GVHD

_____ Ngày: _____

ThS. Mai Đức Trung
Khoa Khoa học và Kỹ thuật Máy tính

Lời cam đoan

Tôi xin cam đoan đây là công trình nghiên cứu và phát triển của riêng tôi dưới sự hướng dẫn của ThS. Mai Đức Trung. Nội dung báo cáo, kết quả triển khai và dữ liệu kiểm thử được trình bày trung thực và chính xác. Ngoài ra, tôi cũng có sử dụng một số nhận xét, đánh giá và số liệu của các tác giả khác, cơ quan tổ chức khác. Tất cả đều có trích dẫn và chú thích nguồn gốc.

Nếu có phát hiện bất kỳ nội dung vi phạm bản quyền hoặc quyền sở hữu trí tuệ, tôi xin hoàn toàn chịu trách nhiệm về nội dung của báo cáo Đồ án tốt nghiệp này và về việc tuân thủ các quy định liên quan đến học thuật, bản quyền và bảo vệ dữ liệu.

Tp. Hồ Chí Minh, ngày _____ tháng _____ năm 2026

Sinh viên

Nguyễn Thành Vinh

Lời cảm ơn

Lời đầu tiên, tôi xin chân thành cảm ơn ThS. Mai Đức Trung đã tận tình hướng dẫn, góp ý và định hướng trong suốt quá trình thực hiện Đồ án tốt nghiệp. Những nhận xét của thầy là cơ sở quan trọng để em hoàn thiện cả sản phẩm phần mềm và báo cáo.

Tôi xin cảm ơn Ban Giám hiệu Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh cùng quý thầy cô Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp nền tảng kiến thức về phát triển phần mềm, cơ sở dữ liệu, hệ thống web và phương pháp giải quyết vấn đề.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn hỗ trợ, động viên và tạo điều kiện để em hoàn thành đồ án tốt nghiệp này.

Tp. Hồ Chí Minh, ngày _____ tháng _____ năm 2026

Sinh viên

Nguyễn Thành Vinh

Tóm tắt nội dung

Xu thế sử dụng trí tuệ nhân tạo (AI) trong các ứng dụng phần mềm đã và đang dần ngày càng trở nên phổ biến hơn trong những năm gần đây. Trong vòng 4 năm kể từ ngày phát hành chính thức, AI đã có những bước tiến vượt bậc. 4 năm trước, không ai nghĩ AI có thể thay thế con người. 4 năm sau, AI đã có thể thay thế con người trong nhiều lĩnh vực, từ những việc đơn giản như trả lời câu hỏi, viết email, đến những việc phức tạp như phân tích dữ liệu, đưa ra quyết định, sáng tạo nội dung.

Đồ án tập trung xây dựng trang web RabbitCV, một ứng dụng web hỗ trợ người dùng tạo, quản lý và trình bày CV theo khổ A4; đồng thời tích hợp trí tuệ nhân tạo để hỗ trợ viết và cải thiện phần giới thiệu, rà soát nội dung, gợi ý bổ sung các mục còn thiếu, đánh giá mức độ phù hợp giữa CV và vị trí việc làm, cũng như luyện tập phỏng vấn.

Giao diện được xây dựng bằng React, Vite và Tailwind CSS; máy chủ sử dụng Node.js, Express và MongoDB/Mongoose. REST API xử lý phần lớn nghiệp vụ, JWT trong cookie HTTP-only duy trì phiên đăng nhập, Socket.IO hỗ trợ nhắn tin, node-cron cập nhật tin hết hạn và PDF.js hiển thị CV PDF theo từng trang. Nhóm chúc năng AI gọi Groq qua API tương thích OpenAI, sử dụng JSON Schema cho đầu ra có cấu trúc, đồng thời áp dụng xác thực, giới hạn tần suất, quota, timeout và làm sạch dữ liệu trước và sau khi gọi mô hình.

MỤC LỤC

1	Tổng quan đề tài	1
1.1	Bối cảnh và lý do chọn đề tài	1
1.2	Mục tiêu của đề tài	1
1.3	Đối tượng sử dụng	2
1.4	Kết quả và giá trị đạt được	2
1.5	Cấu trúc báo cáo	2
2	Cơ sở lý thuyết	4
2.1	Kiến trúc ứng dụng web	4
2.2	MERN Stack	4
2.3	Cơ chế phân quyền RBAC	5
2.4	Lựa chọn công nghệ	5
2.5	Xây dựng và quản lý giao diện người dùng	6
2.6	Socket.IO	6
3	Giới thiệu các công nghệ đã sử dụng	7
3.1	Phát triển giao diện người dùng (Front-end)	7
3.1.1	React	7
3.1.2	TanStack Query	8
3.1.3	React Hook Form và Zod	9
3.1.4	Tailwind CSS	10
3.1.5	Vite	11
3.2	Phát triển Back-end	12
3.2.1	Node.js và Express	12
3.2.2	Socket.IO	12
3.3	Phát triển cơ sở dữ liệu	13
3.3.1	MongoDB	13
3.3.2	Mongoose	14
3.3.3	Sự kết hợp giữa MongoDB và Mongoose	15
3.3.4	Dữ liệu lồng nhau và quan hệ tham chiếu	16
3.3.5	Chỉ mục, ràng buộc và tối ưu dữ liệu	16
3.4	Phát triển trí tuệ nhân tạo	17
3.4.1	Mô hình ngôn ngữ lớn, Groq và OpenAI SDK	17
3.4.2	JSON Schema và đầu ra có cấu trúc	17
3.4.3	Kiểm soát đầu vào, hạn mức và thống kê sử dụng	18
3.5	Phát triển tạo và xuất tệp PDF	18
3.5.1	Bố cục A4 và react-to-print	18
3.5.2	PDF.js và hiển thị PDF tải lên	19
3.5.3	Kiểm tra, lưu trữ và tối ưu dữ liệu PDF	19

4	Phân tích và thiết kế hệ thống	21
4.1	Phân tích tác nhân	21
4.2	Yêu cầu chức năng	22
4.3	Yêu cầu phi chức năng	23
4.4	Ma trận phân quyền	23
4.5	Kiến trúc tổng thể	24
4.6	Thiết kế mô-đun	24
4.7	Thiết kế dữ liệu	24
4.7.1	User và Resume	25
4.7.2	Job và Application	25
4.7.3	Message, Notification và trạng thái đọc	25
4.7.4	AiUsage	25
4.8	Các quy trình nghiệp vụ chính	26
4.8.1	Ứng tuyển bằng CV	26
4.8.2	Yêu cầu AI	26
4.8.3	Chat và trạng thái đọc	26
4.9	Bảo mật và ranh giới tin cậy	26
5	Triển khai hệ thống	28
5.1	Tổ chức mã nguồn và phạm vi triển khai	28
5.2	Xác thực và phân quyền	29
5.3	Tạo, quản lý và xuất CV	29
5.4	Tích hợp AI có kiểm soát	30
5.5	Tìm việc và ứng tuyển	31
5.6	Cổng doanh nghiệp	32
5.7	Chat và thông báo	32
5.8	Khu vực quản trị	32
5.9	Bảo mật và giới hạn triển khai	33
5.10	Cấu hình và khả năng vận hành	33
5.11	Đối chiếu yêu cầu với hiện thực	34
6	Mô-đun trí tuệ nhân tạo	35
6.1	Mục tiêu và phạm vi	35
6.2	Kiến trúc tổng thể của mô-đun	35
6.2.1	Thành phần giao diện và lớp API	35
6.2.2	Bộ làm sạch và dịch vụ AI	36
6.2.3	Lưu thống kê sử dụng	36
6.3	Các chức năng AI	36
6.3.1	Viết và cải thiện phần giới thiệu	36
6.3.2	Rà soát nội dung CV	36
6.3.3	Gợi ý bổ sung các mục còn thiếu	37
6.3.4	Đánh giá mức độ phù hợp giữa CV và công việc	37
6.3.5	Luyện tập phỏng vấn	37
6.4	Quy trình xử lý một yêu cầu AI	37
6.4.1	Xác thực, giới hạn tốc độ và hạn mức	37
6.4.2	Làm sạch và rút gọn dữ liệu	37
6.4.3	Tạo yêu cầu và đầu ra có cấu trúc	37
6.4.4	Trả kết quả và quyền quyết định của người dùng	38
6.5	Quản lý thống kê sử dụng	38
6.6	Kiểm soát rủi ro và bảo vệ dữ liệu	38
6.7	Hạn chế của mô-đun	38
6.8	Tổng kết chương	39

7	Kiểm thử và đánh giá	40
7.1	Mục tiêu và nguyên tắc đánh giá	40
7.2	Môi trường và khả năng tái lập	41
7.3	Kết quả quy trình kiểm tra tự động	41
7.4	Kiểm kê bộ kiểm thử	42
7.5	Truy vết yêu cầu và bằng chứng	43
7.6	Kịch bản kiểm thử thủ công cần thực hiện	44
7.7	Đánh giá riêng cho chức năng AI	44
7.8	Hạn chế và rủi ro còn lại	45
8	Kết luận và hướng phát triển	46
8.1	Mức độ hoàn thành mục tiêu	46
8.2	Đóng góp kỹ thuật	46
8.3	Bài học kinh nghiệm	47
8.4	Hạn chế hiện tại	47
8.5	Lộ trình phát triển	48
8.6	Tổng kết	48
A	Thông tin tái lập và danh mục kỹ thuật	50
A.1	Quy trình tái lập kiểm tra	50
A.2	Biến môi trường	51
A.3	Danh mục API theo miền nghiệp vụ	51
A.4	Bất biến dữ liệu cần bảo vệ	52
A.5	Rubric đề xuất cho chất lượng AI	52
A.6	Checklist trước khi nộp phiên bản chính thức	53

DANH SÁCH CÁC HÌNH VẼ

4.1	Bản đồ tác nhân và nhóm ca sử dụng chính	21
4.2	Kiến trúc triển khai của RabbitCV	24
4.3	Mô hình quan hệ giữa các collection chính	25
4.4	Luồng kiểm soát khi ứng tuyển bằng CV	26
4.5	Đường đi của dữ liệu trong một yêu cầu AI	26
5.1	Giao diện đăng nhập của RabbitCV	29
5.2	Khung chỉnh sửa và vùng preview CV	30
5.3	Giao diện tìm kiếm và xem nhanh việc làm	31
6.1	Kiến trúc xử lý của mô-đun trí tuệ nhân tạo	35

Chương 1

Tổng quan đề tài

1.1 Bối cảnh và lý do chọn đề tài

CV là điểm tiếp xúc đầu tiên giữa nhiều ứng viên và nhà tuyển dụng. Một CV có bố cục rõ ràng, nội dung nhất quán và tập trung vào năng lực liên quan sẽ giúp người đọc nhanh chóng xác định được mức độ phù hợp ban đầu.

Tuy nhiên, đối với người mới tham gia thị trường lao động, họ thường gặp ba khó khăn chính: thiếu kinh nghiệm trong việc tổ chức thông tin, chưa biết cách diễn đạt kinh nghiệm theo ngôn ngữ nghề nghiệp và chưa biết điều chỉnh CV theo vị trí mục tiêu.

Bên cạnh đó, một thực tế đáng quan ngại là nhiều ứng viên vẫn đang sử dụng các phiên bản CV cũ kỹ, thiếu chuyên nghiệp và không tuân thủ các tiêu chuẩn hiện đại, điều này làm giảm đáng kể cơ hội của họ trong thị trường lao động đầy cạnh tranh.

Các công cụ AI có thể hỗ trợ người dùng viết lại, tóm tắt và phân tích nội dung. Tuy nhiên, nội dung do mô hình tạo ra có thể chưa chính xác, vì vậy người dùng cần kiểm tra trước khi sử dụng. Bên cạnh đó, nếu thông tin nhạy cảm được đưa vào hệ thống mà không có biện pháp kiểm soát phù hợp, dữ liệu có nguy cơ bị sử dụng ngoài mục đích ban đầu. Do đó, việc tích hợp mô hình ngôn ngữ không đơn thuần là gửi yêu cầu và nhận kết quả, mà còn phải kiểm soát dữ liệu đầu vào, chuẩn hóa đầu ra và quản lý quyền truy cập. Người dùng vẫn là người xem xét và đưa ra quyết định cuối cùng đối với nội dung do AI đề xuất. Cách tiếp cận này phù hợp với các nguyên tắc quản lý rủi ro AI về tính minh bạch, khả năng kiểm tra và vai trò trung tâm của con người [12].

Từ nhu cầu trên, đề tài lựa chọn xây dựng trang web RabbitCV – ứng dụng web hỗ trợ tạo CV và tìm kiếm việc làm có tích hợp AI. Tạo và cải thiện CV chính là trọng tâm; các chức năng tìm việc, ứng tuyển, doanh nghiệp, thông báo và nhấn tin tạo ra bối cảnh thực tế để người dùng sử dụng CV sau khi hoàn thành.

1.2 Mục tiêu của đề tài

Mục tiêu tổng quát là xây dựng một trang web có thể vận hành để hỗ trợ người dùng tạo CV và các thao tác tuyển dụng liên quan, trong đó AI sẽ đóng vai trò trợ lý hỗ trợ người dùng và không thay thế quyết định của họ. Các mục tiêu cụ thể gồm:

1. Xây dựng giao diện và quy trình tạo, chỉnh sửa, lưu, xem trước và in CV theo tỷ lệ A4.
2. Hỗ trợ tải lên CV định dạng PDF, chỉ chấp nhận dữ liệu PDF hợp lệ và giới hạn kích thước giải mã ở 5 MB.

3. Tích hợp AI viết mới, cải thiện hoặc rút gọn tiểu sử; review CV; gợi ý điền mục còn thiếu; chấm mức độ phù hợp với công việc; và tập luyện phỏng vấn với AI.
4. Cung cấp tìm kiếm, lưu tin, ứng tuyển bằng CV đã lưu hoặc PDF, theo dõi trạng thái và duy trì lịch sử khi tin bị đóng hoặc xóa.
5. Tổ chức quyền truy cập cho khách, ứng viên, doanh nghiệp và quản trị viên; kiểm tra vai trò và quyền sở hữu ở backend.
6. Hỗ trợ chat bằng Socket.IO, lưu trạng thái đọc theo từng người dùng và quản lý thông báo.

1.3 Đối tượng sử dụng

Khách xem nội dung công khai, danh sách và chi tiết công việc. Ứng viên quản lý hồ sơ, tạo hoặc tải CV, dùng công cụ AI, tìm và lưu việc, ứng tuyển, theo dõi trạng thái, nhận thông báo và nhắn tin. Doanh nghiệp quản lý thông tin công ty, tạo và điều chỉnh tin, tiếp nhận hồ sơ, xem CV có liên quan, cập nhật trạng thái và trao đổi với ứng viên. Quản trị viên theo dõi số liệu, quản lý người dùng, doanh nghiệp và tin tuyển dụng.

1.4 Kết quả và giá trị đạt được

Đề tài tạo ra một nguyên mẫu thống nhất giữa soạn CV và bối cảnh tuyển dụng. Những điểm kỹ thuật có giá trị gồm:

- Phân biệt CV được tạo trực tiếp trên hệ thống với CV PDF tải lên bằng một trường đánh dấu loại CV; đồng thời chỉ trả về các thông tin cần thiết khi hiển thị danh sách CV, còn nội dung PDF đầy đủ được tải khi người dùng mở hoặc xem CV cụ thể.
- Lưu bản chụp thông tin việc làm tại thời điểm ứng tuyển cùng hồ sơ ứng tuyển, giúp duy trì ngữ cảnh lịch sử ngay cả khi tin tuyển dụng bị xóa.
- Thiết lập ràng buộc duy nhất cho mỗi cặp ứng viên và tin tuyển dụng, qua đó ngăn ngừa việc gửi hồ sơ trùng lặp ngay tại tầng cơ sở dữ liệu.
- Lưu trạng thái đã đọc và trạng thái ẩn của cuộc trò chuyện riêng cho từng người dùng thay vì xóa dữ liệu dùng chung.
- Xây dựng năm nhóm chức năng AI với kiểm soát đầu vào, đầu ra, giới hạn hạn mức và tần suất sử dụng, kiểm soát thời gian xử lý và thống kê mức độ sử dụng.

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành tám chương và một phần phụ lục. Nội dung, trọng tâm và điểm nổi bật của từng phần được trình bày như sau:

- **Chương 1 – Tổng quan đề tài:** Trình bày bối cảnh và lý do lựa chọn đề tài, mục tiêu xây dựng hệ thống, các nhóm người dùng, kết quả đạt được và giá trị của sản phẩm. Chương này xác định vai trò hỗ trợ của AI, trong đó người dùng vẫn kiểm tra và quyết định cuối cùng đối với nội dung được đề xuất.
- **Chương 2 – Cơ sở lý thuyết:** Trình bày các kiến thức nền tảng liên quan đến kiến trúc ứng dụng web, tổ chức hệ thống, quản lý dữ liệu, xác thực, phân quyền, giao tiếp theo thời gian thực, xử lý PDF và mô hình ngôn ngữ.

- **Chương 3 – Giới thiệu các công nghệ đã sử dụng:** Giới thiệu các công nghệ được dùng để phát triển giao diện, máy chủ, cơ sở dữ liệu, trí tuệ nhân tạo, xử lý PDF, đồng bộ dữ liệu, bảo mật, kiểm thử và triển khai RabbitCV.
- **Chương 4 – Phân tích và thiết kế hệ thống:** Chuyển các mục tiêu của đề tài thành các nhóm tác nhân, yêu cầu chức năng, yêu cầu phi chức năng, ma trận phân quyền, kiến trúc tổng thể, thiết kế dữ liệu và các quy trình nghiệp vụ chính. Chương này nhấn mạnh mối liên hệ giữa yêu cầu, quyền truy cập, dữ liệu và ranh giới bảo mật trong từng luồng sử dụng.
- **Chương 5 – Triển khai hệ thống:** Trình bày quá trình hiện thực các chức năng chính gồm xác thực, tạo và quản lý CV, xuất CV, tìm việc, ứng tuyển, quản lý phía doanh nghiệp, nhắn tin và thông báo.
- **Chương 6 – Mô-đun trí tuệ nhân tạo:** Giải thích kiến trúc mô-đun AI, năm chức năng đã triển khai, quy trình xử lý yêu cầu, kiểm soát dữ liệu, thống kê sử dụng, giới hạn hiện tại và vai trò quyết định cuối cùng của người dùng.
- **Chương 7 – Kiểm thử và đánh giá:** Trình bày môi trường đánh giá, khả năng tái lập, kết quả kiểm tra tự động và thủ công, danh mục kiểm thử, truy vết yêu cầu và các kịch bản kiểm thử thủ công. Chương có điểm nhấn là phân biệt rõ bằng chứng kiểm thử nghiệp vụ, kiểm tra kỹ thuật và đánh giá riêng cho chức năng AI.
- **Chương 8 – Kết luận và hướng phát triển:** Đối chiếu kết quả thực hiện với các mục tiêu ban đầu, tổng hợp đóng góp kỹ thuật, bài học kinh nghiệm, hạn chế hiện tại và lộ trình phát triển. Điểm nổi bật của chương là nêu rõ những giới hạn chưa được kiểm chứng, đặc biệt đối với chất lượng ngữ nghĩa của kết quả AI.
- **Phụ lục – Thông tin tái lập và danh mục kỹ thuật:** Cung cấp quy trình tái lập kiểm tra, các biến môi trường, danh mục giao tiếp của hệ thống, những nguyên tắc bảo toàn dữ liệu, tiêu chí đánh giá chất lượng AI và danh sách kiểm tra trước khi nộp báo cáo.

Nhìn tổng thể, báo cáo được xây dựng theo mạch từ bối cảnh và mục tiêu, đến cơ sở lý thuyết, công nghệ sử dụng, phân tích thiết kế, triển khai, mô-đun AI, kiểm thử và kết luận. Cách tổ chức này giúp người đọc theo dõi được quá trình chuyển đổi từ yêu cầu ban đầu thành hệ thống, đồng thời đối chiếu được kết quả với các bằng chứng kiểm tra tương ứng.

Chương 2

Cơ sở lý thuyết

2.1 Kiến trúc ứng dụng web

RabbitCV được triển khai theo kiến trúc client–server dạng monolith module.

Ở phía client, React và React Router tổ chức giao diện theo các nhóm trang công khai, ứng viên, doanh nghiệp và quản trị. TanStack Query quản lý dữ liệu lấy từ server bằng cache, truy vấn lại và cập nhật lạc quan. React Hook Form kết hợp Zod kiểm tra dữ liệu biểu mẫu trước khi gửi. Vite đảm nhiệm máy chủ phát triển, proxy các đường dẫn /api và /socket.io đến backend, đồng thời chia frontend production thành các nhóm thư viện như PDF, rich text, query, form và React để giảm lượng mã tải ban đầu.

Ở phía server, Node.js và Express tập trung vào trong server/server.js, sau đó chuyển tiếp đến các route nghiệp vụ, service AI, schema đầu ra, sanitizer, model Mongoose và các hàm xử lý dùng chung. Chuỗi middleware đặt trước nghiệp vụ gồm CORS, Helmet, cookie parser, giới hạn tốc độ và xác thực JWT. Backend là nơi quyết định cuối cùng về phiên đăng nhập, vai trò, quyền sở hữu dữ liệu, giới hạn request và tính hợp lệ của payload.

2.2 MERN Stack

MERN Stack là một nhóm công nghệ dùng để xây dựng ứng dụng web toàn ngăn xếp, gồm MongoDB, Express, React và Node.js. MongoDB đảm nhiệm lưu trữ dữ liệu dưới dạng tài liệu; Express cung cấp cơ chế định tuyến và middleware cho máy chủ; React xây dựng giao diện người dùng theo các component; còn Node.js cung cấp môi trường chạy JavaScript phía máy chủ.

Bốn thành phần có thể phối hợp qua REST API và dữ liệu JSON, giúp quá trình phát triển giữa frontend và backend có sự thống nhất về ngôn ngữ và cách trao đổi dữ liệu:

- React đảm nhiệm phần giao diện và các luồng tương tác;
- Node.js và Express xử lý REST API, xác thực, phân quyền và nghiệp vụ;
- MongoDB kết hợp với Mongoose để lưu trữ người dùng, CV, tin tuyển dụng, đơn ứng tuyển, tin nhắn và các dữ liệu liên quan.

MERN Stack phù hợp với đề tài vì hỗ trợ phát triển nhanh ứng dụng web có dữ liệu thay đổi thường xuyên, cấu trúc CV lồng nhau và các chức năng cần giao tiếp giữa trình duyệt với máy chủ.

2.3 Cơ chế phân quyền RBAC

RBAC (Role-Based Access Control) là cơ chế kiểm soát truy cập dựa trên vai trò. Thay vì cấp quyền riêng lẻ cho từng tài khoản, hệ thống gán cho người dùng một vai trò và xác định các thao tác mà vai trò đó được phép thực hiện. Cách tổ chức này giúp chính sách quyền dễ mô tả, dễ kiểm tra và hạn chế việc lặp lại các điều kiện phân quyền trong từng chức năng.

RabbitCV tổ chức người dùng thành bốn vai trò chính:

- Khách: khi chưa đăng nhập chỉ được truy cập các nội dung công khai.
- Ứng viên: khi đăng nhập được quản lý CV cá nhân, lưu việc, ứng tuyển và sử dụng các chức năng hỗ trợ AI.
- Doanh nghiệp: khi đăng nhập được tạo và quản lý tin tuyển dụng của mình, xem các hồ sơ liên quan đến tin đã đăng và cập nhật trạng thái xử lý ứng tuyển.
- Quản trị viên: khi đăng nhập được sử dụng các chức năng quản trị người dùng, doanh nghiệp và tin tuyển dụng.

Khi đăng nhập, vai trò được đưa vào thông tin phiên và token JWT. Backend xác minh token trước khi xử lý API riêng tư, sau đó kiểm tra vai trò đối với các thao tác đặc quyền. Với dữ liệu có phạm vi sở hữu, kiểm tra vai trò được kết hợp với kiểm tra quan hệ giữa người dùng và dữ liệu: doanh nghiệp chỉ thao tác trên tin do mình quản lý, ứng viên chỉ chỉnh sửa CV của mình, còn quản trị viên có quyền quản trị theo chính sách hệ thống. Vì vậy, RabbitCV sử dụng RBAC kết hợp với kiểm tra quyền sở hữu và ngữ cảnh nghiệp vụ, thay vì chỉ dựa vào việc người dùng đã đăng nhập.

2.4 Lựa chọn công nghệ

Công nghệ	Vai trò	Lý do trong RabbitCV
React, React Router	Component, state và điều hướng.	Giao diện có nhiều biểu mẫu CV, vùng xem trước, cổng theo vai trò và trang được tải lười.
Vite, Tailwind CSS, Radix UI, Sonner	Phát triển, đóng gói và nền tảng giao diện.	Chia gói mã theo nhóm; thống nhất bố cục, hộp thoại, danh sách chọn, menu và phản hồi thao tác.
Node.js, Express	REST API và middle-ware.	Dùng JavaScript xuyên suốt, thuận tiện chia sẻ định dạng JSON và xử lý dữ liệu bất đồng bộ.
MongoDB, Mon-goose	Lưu tài liệu và mô hình dữ liệu.	CV có nhiều mảng lồng nhau; schema và index giúp ràng buộc các quy tắc quan trọng.
Socket.IO	Nhắn tin bằng sự kiện.	Hỗ trợ room, xác thực khi bắt tay và kết nối lại ở phía client.
react-to-print, PDF.js	Xuất và kết xuất CV PDF.	Dùng chung quy tắc in A4 cho CV web và hiển thị từng trang PDF tải lên thay vì phụ thuộc trình xem gốc.
Groq, OpenAI SDK	Gọi mô hình ngôn ngữ.	API tương thích OpenAI và hỗ trợ đầu ra theo JSON Schema cho các luồng AI.

Bảng 2.1: Vai trò và lý do lựa chọn công nghệ

2.5 Xây dựng và quản lý giao diện người dùng

RabbitCV sử dụng React để tổ chức giao diện. Các trang giao diện được chia theo nhóm chức năng gồm: trang công khai, trang ứng viên, trang doanh nghiệp và trang quản trị. Những thành phần chung như thanh điều hướng, biểu mẫu, hộp xác nhận, thông báo và vùng xem trước được tách riêng để hạn chế lặp mã và duy trì cách trình bày thống nhất.

Thông tin phiên đăng nhập, danh sách tin nhắn và các hộp thoại dùng chung được cung cấp cho những trang liên quan thông qua React Context. React Router quản lý đường dẫn và nhóm trang của hệ thống [10].

TanStack Query được sử dụng để quản lý dữ liệu lấy từ máy chủ thông qua khóa truy vấn, bộ nhớ đệm, trạng thái tải, trạng thái lỗi và cơ chế làm mới dữ liệu [14]. Việc lưu và tái sử dụng dữ liệu theo từng khóa giúp hạn chế các yêu cầu lặp, đồng thời giúp giao diện phản hồi nhanh hơn sau thao tác của người dùng.

Bên cạnh đó, cơ chế *lazy load* được áp dụng cho mã nguồn frontend của các trang lớn thông qua React.lazy và Suspense. Nhờ đó, mã của một trang chỉ được tải khi người dùng điều hướng đến trang đó, góp phần giảm dung lượng phải tải ở lần truy cập đầu tiên.

React Hook Form kết hợp với Zod để kiểm tra dữ liệu biểu mẫu ở phía trình duyệt, hiển thị lỗi theo từng trường và chỉ gửi yêu cầu khi dữ liệu cơ bản hợp lệ. Việc tách dữ liệu máy chủ khỏi trạng thái trình bày, đồng thời tách mã nguồn theo từng trang, giúp mã dễ theo dõi, giảm yêu cầu không cần thiết và hạn chế tình trạng các trang sử dụng những bản dữ liệu không đồng nhất.

2.6 Socket.IO

Socket.IO là một thư viện JavaScript cho phép giao tiếp hai chiều (bidirectional), dựa trên sự kiện (event-based) và theo thời gian thực (real-time) giữa trình duyệt (Client) và máy chủ (Server) với độ trễ cực thấp. Với Socket.IO, Server có thể chủ động đẩy (push) dữ liệu xuống Client ngay lập tức khi có dữ liệu mới.

Trong RabbitCV, Socket.IO được tích hợp chặt chẽ với ứng dụng web để xử lý các tác vụ yêu cầu cập nhật ngay lập tức. Khi một người dùng đăng nhập, một kết nối socket được thiết lập giữa client và server. Server quản lý các kết nối này bằng cách phân loại chúng vào các "phòng" (rooms) riêng biệt để gửi dữ liệu đến chính xác người dùng cần nhận, mà không ảnh hưởng đến những người dùng khác.

Socket.IO trong dự án này chỉ phục vụ tin nhắn trò chuyện theo thời gian thực.

Thông báo nghiệp vụ không được đẩy qua Socket.IO mà được lấy từ các endpoint thông báo bằng fetch; TanStack Query quản lý bộ nhớ đệm, phân trang, trạng thái đã đọc và làm mới số thông báo chưa đọc theo chu kỳ 30 giây. Cách tách này giúp phân biệt rõ kênh sự kiện thời gian thực với cơ chế đồng bộ dữ liệu định kỳ.

Chương 3

Giới thiệu các công nghệ đã sử dụng

3.1 Phát triển giao diện người dùng (Front-end)

3.1.1 React

React là thư viện JavaScript mã nguồn mở được sử dụng để xây dựng giao diện người dùng theo mô hình thành phần [9]. Thay vì viết toàn bộ giao diện trong một khối lớn, React cho phép chia trang thành các component nhỏ như thanh điều hướng, biểu mẫu, danh sách, hộp thoại và khu vực xem trước. Mỗi component đảm nhiệm một nhiệm vụ cụ thể, có thể nhận dữ liệu đầu vào và được kết hợp với các component khác để hình thành một trang hoàn chỉnh. Cách tổ chức này giúp cấu trúc giao diện rõ ràng hơn và phù hợp với những ứng dụng có nhiều khu vực chức năng như RabbitCV.

Các tính năng chính của React bao gồm:

- **Kiến trúc dựa trên component:** Giao diện được chia thành các thành phần độc lập, trong đó mỗi component quản lý một phần nội dung và hành vi cụ thể. Component có thể được kết hợp, tái sử dụng và mở rộng để tạo thành những trang có cấu trúc phức tạp. Cách tổ chức này giúp tách biệt trách nhiệm giữa các phần giao diện và làm mã nguồn dễ theo dõi hơn.
- **Quản lý dữ liệu của component:** React phân biệt dữ liệu được truyền vào và dữ liệu thay đổi bên trong component. Các thành phần chính gồm:
 - **Props:** Là dữ liệu được truyền từ component cha xuống component con. Props giúp cùng một component có thể hiển thị nội dung hoặc thực hiện hành vi khác nhau tùy theo dữ liệu nhận được.
 - **State:** Là dữ liệu có thể thay đổi trong quá trình người dùng tương tác. Khi state thay đổi, React thực hiện kết xuất lại những phần giao diện liên quan để phản ánh trạng thái mới.
 - **Luồng dữ liệu một chiều:** Dữ liệu thường được truyền từ component cha xuống component con thông qua props. Component con không trực tiếp thay đổi dữ liệu của component cha. Cơ chế này giúp xác định nguồn của dữ liệu và theo dõi nguyên nhân làm giao diện thay đổi rõ ràng hơn.
- **Cơ chế cập nhật giao diện hiệu quả:** Khi props hoặc state thay đổi, React xây dựng lại giao diện và so sánh với kết quả trước đó. Quá trình đối chiếu giúp xác định những phần cần thay đổi trên DOM thực, nhờ đó tránh cập nhật toàn bộ trang khi chỉ một vùng dữ liệu thay đổi. Người dùng có thể tiếp tục tương tác mà không phải tải lại trang sau mỗi thao tác.

- **Hệ thống Hook:** là tập hợp các hàm cho phép component dạng hàm sử dụng các khả năng của React như lưu trạng thái, đồng bộ với hệ thống bên ngoài, truy cập dữ liệu dùng chung và tái sử dụng logic. Hook giúp React trở nên linh hoạt và dễ sử dụng hơn. Một số Hook phổ biến gồm:
 - **useState:** Lưu và cập nhật trạng thái cục bộ của component.
 - **useEffect:** Thực hiện tác vụ sau khi component được hiển thị hoặc khi dữ liệu phụ thuộc thay đổi.
 - **useMemo và useCallback:** Lưu lại kết quả tính toán hoặc hàm khi cần thiết, qua đó hạn chế việc tạo lại không cần thiết trong một số trường hợp.

React mang lại một số ưu điểm nổi bật như sau:

- **Khả năng tái sử dụng cao:** Một component biểu mẫu, nút thao tác, thẻ việc làm hoặc vùng hiển thị CV có thể được sử dụng tại nhiều trang với dữ liệu khác nhau. Điều này giảm mã lặp và giúp thay đổi giao diện được thực hiện tập trung.
- **Dễ tổ chức và bảo trì:** Việc chia giao diện thành các component có trách nhiệm rõ ràng giúp nhà phát triển dễ tìm kiếm, sửa đổi và mở rộng từng chức năng mà ít ảnh hưởng đến những phần không liên quan.
- **Giao diện phản hồi nhanh:** React chỉ cập nhật những vùng bị ảnh hưởng bởi thay đổi dữ liệu, giúp các thao tác như nhập biểu mẫu, mở hộp thoại, lọc danh sách và thay đổi bản xem trước diễn ra mà không tải lại toàn bộ trang.
- **Hệ sinh thái phong phú:** React có cộng đồng lớn và nhiều thư viện hỗ trợ biểu mẫu, truy vấn dữ liệu, trình soạn thảo, hiển thị PDF và các nhu cầu khác của ứng dụng web. Dự án có thể lựa chọn công cụ phù hợp thay vì tự xây dựng toàn bộ chức năng từ đầu.

Trong RabbitCV, React được dùng để xây dựng các trang công khai, khu vực ứng viên, cổng doanh nghiệp và khu vực quản trị. Trình biên tập CV tận dụng trạng thái component để phản ánh dữ liệu người dùng nhập vào bản xem trước; danh sách việc làm và hồ sơ ứng tuyển cập nhật theo dữ liệu nhận từ máy chủ; các hộp thoại và thông báo thay đổi theo thao tác mà không tải lại trang. Việc tổ chức giao diện thành component giúp dự án có thể dùng chung các thành phần, mở rộng chức năng theo từng khu vực và duy trì trải nghiệm nhất quán trên toàn hệ thống.

3.1.2 TanStack Query

TanStack Query là thư viện quản lý dữ liệu máy chủ trong ứng dụng web, giúp đơn giản hóa việc lấy dữ liệu (fetching), lưu bộ nhớ đệm (caching), đồng bộ hóa (synchronizing) và cập nhật dữ liệu bất đồng bộ trong các ứng dụng web. Trong RabbitCV, TanStack Query giúp các thành phần giao diện sử dụng dữ liệu máy chủ theo một cơ chế thống nhất, hạn chế yêu cầu trùng lặp và giảm lượng mã xử lý thủ công. Các ưu điểm chính của TanStack Query:

- **Quản lý bộ nhớ đệm:** Lưu và dùng chung dữ liệu giữa các thành phần, giúp hạn chế yêu cầu API trùng lặp.
- **Quản lý trạng thái yêu cầu:** Cung cấp sẵn trạng thái đang tải, thành công, thất bại và đang cập nhật dữ liệu.
- **Làm mới và đồng bộ dữ liệu:** Cho phép cập nhật đúng nhóm dữ liệu liên quan mà không cần tải lại toàn bộ trang.
- **Hỗ trợ phân trang:** Giữ nội dung hiện tại trong lúc tải trang tiếp theo, giúp giao diện ổn định hơn.
- **Cập nhật lạc quan:** Hiển thị trước kết quả thao tác và khôi phục dữ liệu cũ nếu yêu cầu thất bại.

- **Giảm mã xử lý thủ công:** Hạn chế việc tự xây dựng bộ nhớ đệm, trạng thái tải, xử lý lỗi và đồng bộ dữ liệu.

Sự khác biệt giữa TanStack Query với `fetch` và `Axios`

TanStack Query và `Axios`, `fetch` là 2 nhóm giải quyết công việc khác nhau. `fetch` và `Axios` thuộc lớp giao tiếp mạng: chúng tạo và gửi yêu cầu HTTP, truyền tham số, phương thức và nội dung yêu cầu đến máy chủ, sau đó nhận phản hồi HTTP. Còn TanStack Query thuộc lớp quản lý dữ liệu trên giao diện: lưu, theo dõi, quản lý bộ nhớ đệm, xác định dữ liệu và cung cấp trạng thái cho component.

TanStack Query không thể trực tiếp tạo hay gửi yêu cầu HTTP đến máy chủ. Để thực hiện điều này, nó cần được cung cấp thông qua các hàm bất đồng bộ trả về Promise. Trong ứng dụng sử dụng REST API, hàm này thường dùng `fetch` hoặc `Axios` để xác định địa chỉ API, phương thức HTTP, tiêu đề, cookie và nội dung yêu cầu; đồng thời nhận, chuyển đổi và kiểm tra phản hồi của máy chủ. Sau đó, TanStack Query tiếp nhận kết quả để xử lý, vì vậy, `fetch` hoặc `Axios` chịu trách nhiệm truyền dữ liệu giữa trình duyệt và máy chủ, còn TanStack Query chịu trách nhiệm quản lý dữ liệu trong ứng dụng.

Ngược lại, chỉ sử dụng `fetch` hoặc `Axios` cũng chỉ có thể gửi yêu cầu và trả kết quả nhưng lại không tự quản lý bộ nhớ đệm dùng chung cho React, không xác định dữ liệu còn mới hay đã cũ, không tự đồng bộ các component đang dùng cùng một nguồn dữ liệu và không tự làm mới những nhóm dữ liệu bị ảnh hưởng sau thao tác cập nhật. Do đó, quan hệ giữa chúng là bổ sung: `fetch` hoặc `Axios` thực hiện việc truyền dữ liệu qua HTTP, còn TanStack Query quản lý dữ liệu sau khi yêu cầu được tổ chức trong ứng dụng.

3.1.3 React Hook Form và Zod

React Hook Form là một thư viện quản lý form nhẹ, nhanh và được sử dụng rộng rãi trong cộng đồng React. Nó giúp giảm thiểu lượng code phức tạp, đồng thời cải thiện hiệu năng và khả năng mở rộng cho ứng dụng.

Một vài ưu điểm nổi bật của React Hook Form có thể kể đến như:

- **Hiệu năng cao:** Bằng cách tận dụng cơ chế "uncontrolled components", nó giúp hạn chế tối đa việc re-render lại toàn bộ dữ liệu mỗi khi người dùng nhập dữ liệu, từ đó giảm thiểu lượng code phức tạp và cải thiện hiệu năng.
- **Ít code thủ công hơn:** Với việc không cần sử dụng hay tạo ra các state và hàm xử lý riêng lẻ cho từng trường dữ liệu, React Hook Form giúp tối ưu hóa quy trình code, giảm thiểu code và tránh các lỗi phát sinh từ việc quản lý state thủ công.
- **Hỗ trợ validation mạnh mẽ:** React Hook Form có hỗ trợ sẵn các tính năng cơ bản trong việc validation, ví dụ như xác thực required, min, max,.... Đồng thời, nó cũng tích hợp mượt mà với các thư viện kiểm tra schema hàng đầu như Zod, Yup, Valibot.
- **Hỗ trợ form phức tạp:** React Hook Form cung cấp đầy đủ các tính năng cần thiết để xử lý các form phức tạp, bao gồm validation, error handling, form submission, và form reset. Nó giúp xử lý tốt input động, dữ liệu lồng nhau, form nhiều bước và các giá trị mặc định.

Zod

Zod là thư viện khai báo và kiểm tra cấu trúc dữ liệu bằng schema, có thể sử dụng trong cả JavaScript và TypeScript [16]. Nó giúp bảo đảm dữ liệu nhận được có đúng kiểu, đúng định dạng và đáp ứng các điều kiện của ứng dụng hay không. Đồng thời, nó giúp xây dựng các schema mô tả format dữ liệu theo định dạng mong muốn, sau đó dùng schema này để kiểm tra dữ liệu thực tế. Nếu dữ liệu không khớp với schema, Zod sẽ cho biết lỗi sai ở đâu.

Zod cũng có một vài ưu điểm đáng chú ý:

- **Kiểm tra dữ liệu khi chương trình đang chạy:** Trước đây việc kiểm tra dữ liệu thường chỉ được thực hiện khi chương trình đang trong quá trình phát triển. Tuy nhiên khi chương trình được triển khai và có người dùng tương tác trực tiếp, việc kiểm tra dữ liệu này trở nên khó khăn hơn. Với Zod, nhà phát triển có thể kiểm tra dữ liệu ngay cả khi chương trình đang chạy, giúp phát hiện lỗi sớm và tăng trải nghiệm người dùng.
- **Hỗ trợ nhiều kiểu dữ liệu:** Zod cung cấp các phương thức để kiểm tra nhiều kiểu dữ liệu khác nhau, bao gồm `string`, `number`, `boolean`, `array`, `object`, `date`, `enum`, và nhiều hơn nữa. Điều này giúp nhà phát triển có thể kiểm tra dữ liệu theo nhiều cách khác nhau, phù hợp với nhu cầu sử dụng của chính mình.
- **Có thể chuyển đổi dữ liệu:** Zod không chỉ kiểm tra mà còn có thể chuẩn hóa dữ liệu đầu vào theo đúng định dạng mà ứng dụng mong muốn.
- **Thông báo lỗi chi tiết:** Khi kiểm tra thất bại, Zod trả về các lỗi có cấu trúc rõ ràng, bao gồm thông tin về trường bị lỗi, loại lỗi và thông tin lỗi chi tiết. Điều này giúp lớp giao diện dễ dàng xác định chính xác trường nào bị lỗi và hiển thị thông báo tương ứng.
- **Hệ sinh thái tích hợp cực kỳ phong phú:** Zod được hỗ trợ bởi nhiều thư viện biểu mẫu, công cụ xây dựng API, cơ sở dữ liệu, sinh tài liệu và kiểm thử. Khả năng này cho phép schema được kết nối hoặc tái sử dụng trong nhiều tầng của ứng dụng, qua đó giảm mã kiểm tra trùng lặp và giúp quy tắc dữ liệu nhất quán hơn.

Lý do kết hợp React Hook Form với Zod

React Hook Form và Zod đảm nhiệm hai vai trò khác nhau nhưng bổ sung trực tiếp cho nhau. React Hook Form quản lý các trường nhập, thu thập giá trị, theo dõi trạng thái biểu mẫu và điều phối thao tác gửi; trong khi Zod tập trung mô tả cấu trúc và kiểm tra tính hợp lệ của dữ liệu. Hai thư viện được kết nối thông qua `zodResolver`, giúp kết quả kiểm tra của Zod được chuyển thành lỗi theo từng trường mà React Hook Form có thể hiển thị [8]. Sự kết hợp giữa React Hook Form và Zod thông qua `zodResolver` mang lại giải pháp quản lý biểu mẫu vừa đạt hiệu năng cao, giảm thiểu việc re-render, vừa đảm bảo dữ liệu đầu vào của ứng dụng luôn chính xác và đáng tin cậy. RabbitCV tận dụng sự kết hợp này để xử lý các form phức tạp trong ứng dụng.

Việc kết hợp mang lại các lợi ích chính sau:

- **Phân chia trách nhiệm rõ ràng:** React Hook Form quản lý trạng thái và tương tác của biểu mẫu, còn Zod quản lý quy tắc và cấu trúc dữ liệu. Điều này giúp code không bị lộn xộn, dễ dàng quản lý và bảo trì.
- **Phản hồi lỗi chính xác:** Lỗi được gắn với đúng trường nhập, giúp người dùng dễ nhận biết và sửa dữ liệu. Người dùng nhận thông báo ngay tại trường nhập liệu bị lỗi thay vì đợi gửi toàn bộ biểu mẫu rồi mới nhận thông báo.
- **Giảm mã lặp và tăng khả năng bảo trì:** Nhà phát triển không phải tạo state, hàm thay đổi và câu lệnh kiểm tra riêng cho từng trường. Quy tắc được đọc tập trung trong schema, còn mã giao diện tập trung vào hiển thị thông báo lỗi cho người dùng.
- **Thống nhất dữ liệu đầu ra:** Hàm xử lý gửi nhận một đối tượng đã vượt qua cùng một bộ quy tắc, giúp giảm các trường hợp mỗi vị trí trong giao diện hiểu dữ liệu theo một cách khác nhau.

3.1.4 Tailwind CSS

Tailwind CSS là framework CSS mã nguồn mở theo hướng tiện ích, giúp xây dựng giao diện chỉ bằng cách kết hợp các lớp tiện ích nhỏ [13]. Thay vì viết bằng CSS truyền thống các lớp

trình viên phải tạo tên lớp riêng rồi khai báo các thuộc tính tương ứng trong tệp CSS. Tailwind cho phép kết hợp trực tiếp các lớp tiện ích ngay trên phần tử giao diện. Việc sử dụng hệ thống lớp tiện ích này giúp RabbitCV dễ dàng duy trì hệ thống khoảng cách, màu sắc và bố cục đáp ứng giữa trang công khai, khu vực ứng viên, doanh nghiệp và quản trị.

Các thành phần tương tác phức tạp được hỗ trợ bởi Radix UI, trong khi Heroicons cung cấp bộ biểu tượng có phong cách thống nhất. Radix UI tập trung vào những nguyên mẫu như hộp thoại, menu và danh sách chọn, đồng thời hỗ trợ quản lý tiêu điểm, bàn phím và các thuộc tính trợ năng. Sonner được dùng để hiển thị thông báo ngắn sau thao tác thành công hoặc thất bại. Việc sử dụng các thư viện chuyên biệt giúp giảm lượng mã tự xây dựng cho những hành vi giao diện dễ phát sinh lỗi và tạo trải nghiệm nhất quán hơn.

Những ưu điểm nổi bật có thể kể đến của Tailwind CSS như:

- **Tối ưu CSS đầu ra:** Tailwind CSS quét mã nguồn và chỉ tạo các utility class thực sự được sử dụng. Khi build production, CSS còn được nén lại, giúp giảm dung lượng tệp, loại bỏ CSS dư thừa và cải thiện tốc độ tải trang. Tailwind tạo CSS tĩnh nên không cần JavaScript xử lý giao diện lúc chạy.
- **Tăng tốc độ phát triển:** Tailwind CSS giúp giảm thời gian phát triển giao diện do có các lớp tiện ích có sẵn.
- **Duy trì giao diện nhất quán:** Các màu sắc, khoảng cách, cỡ chữ, độ bo góc và bóng đổ được lấy từ một hệ thống giá trị chung. Nhờ đó, các thành phần khác nhau có thể duy trì cùng ngôn ngữ thiết kế.
- **Dễ dàng bảo trì và mở rộng:** Các lớp tiện ích giúp dễ dàng thay đổi hoặc thêm các lớp mới mà không ảnh hưởng đến các phần khác của ứng dụng.
- **Hệ sinh thái và công cụ hỗ trợ mạnh mẽ:** Tailwind CSS tích hợp tốt với các thư viện UI và nhiều framework khác nhau, cho phép xây dựng giao diện phức tạp một cách nhanh chóng.
- **Thân thiện với người dùng:** Tailwind CSS có cấu trúc rõ ràng, dễ hiểu, giúp lập trình viên nhanh chóng làm quen và sử dụng. Đồng thời, Tailwind CSS cũng hỗ trợ nhiều tính năng nâng cao như responsive design, dark mode, animation, giúp tạo ra giao diện đẹp và thân thiện với người dùng.

3.1.5 Vite

Vite là một công cụ phát triển cho ứng dụng web frontend hiện đại, được thiết kế để mang đến trải nghiệm phát triển nhanh chóng và hiệu quả, hỗ trợ toàn diện cho React, Vue, Svelte, và các framework khác [15]. Trong môi trường phát triển, Vite cung cấp máy chủ dựa trên ES Module và cơ chế thay thế mô-đun nhanh, giúp cập nhật phần mã vừa thay đổi mà không phải tải lại toàn bộ ứng dụng.

Vite đảm nhận 2 công việc chính: cung cấp máy chủ phát triển để lập trình viên chạy và chỉnh sửa giao diện trên máy tính cá nhân và đóng gói mã nguồn thành các tệp HTML, CSS, JavaScript và tài nguyên tĩnh để triển khai trên máy chủ. Trong RabbitCV, React xây dựng giao diện, Tailwind CSS định dạng giao diện, còn Vite chịu trách nhiệm chạy môi trường phát triển và đóng gói phần frontend.

Ngoài ra, RabbitCV cũng cấu hình Vite để xử lý các yêu cầu API và kết nối thời gian thực. Các yêu cầu được chuyển tiếp đến máy chủ Express thông qua các proxy `/api` và `/socket.io`. Điều này cho phép frontend và backend có thể được phát triển và chạy một cách độc lập, trong khi giao diện người dùng vẫn có thể gọi các đường dẫn thống nhất mà không cần phải lo lắng về việc thiết lập các cấu hình phức tạp.

RabbitCV kết hợp React.lazy với Suspense để tải lười các trang có kích thước lớn. Mã của trang quản trị, cổng doanh nghiệp, quản lý CV hoặc các chức năng chuyên biệt chỉ được tải khi người dùng điều hướng đến đường dẫn tương ứng. Trong thời gian chờ, Suspense hiển thị trạng thái tải thay vì để giao diện trống. Cách làm này làm giảm lượng JavaScript cần tải và phân tích ở lần truy cập đầu tiên, đặc biệt có ích khi một người dùng chỉ sử dụng một nhóm chức năng theo vai trò của mình.

Tải lười không làm giảm tổng lượng mã của toàn bộ hệ thống mà nó thay đổi thời điểm tải mã. Tuy nhiên, dự án áp dụng tải lười ở cấp trang và nhóm chức năng lớn, đồng thời gom những thư viện có quan hệ thành các gói phù hợp để cân bằng giữa tốc độ khởi động, khả năng lưu đệm và độ trễ khi chuyển trang.

3.2 Phát triển Back-end

3.2.1 Node.js và Express

Node.js là nền tảng phát triển các ứng dụng web back-end được viết bằng JavaScript [7]. Khi một thao tác phải chờ cơ sở dữ liệu, hệ thống tệp hoặc dịch vụ AI, Node.js có thể tiếp tục tiếp nhận công việc khác thay vì chặn toàn bộ luồng xử lý. Với việc sử dụng kiến trúc lập trình hướng sự kiện không đồng bộ, nó giúp cho các ứng dụng Node.js có thể chạy với hiệu suất cao và khả năng mở rộng tốt.

Express là framework xây dựng máy chủ web trên Node.js, cung cấp cơ chế định tuyến, xử lý yêu cầu-phản hồi và các tính năng cần thiết cho ứng dụng web [6]. RabbitCV sử dụng Express để tổ chức API theo nhóm tài nguyên và nghiệp vụ, bao gồm tài khoản, CV, việc làm, ứng tuyển, trò chuyện, thông báo và trí tuệ nhân tạo. Mỗi đường dẫn có thể gắn các bước xác thực, kiểm tra vai trò, giới hạn tốc độ và kiểm tra dữ liệu trước khi gọi hàm xử lý chính.

Node.js và Express cho phép frontend và backend cùng sử dụng hệ sinh thái JavaScript, nhờ đó cách biểu diễn JSON, mô hình bất đồng bộ và công cụ quản lý gói được thống nhất. Express có cấu trúc tối giản nên dự án có thể lựa chọn thư viện phù hợp cho cookie, bảo mật, giới hạn tốc độ, cơ sở dữ liệu và thời gian thực. Máy chủ HTTP của Express còn được dùng chung với Socket.IO, tránh phải vận hành một dịch vụ riêng chỉ để truyền sự kiện trò chuyện.

Node.js và Express có các đặc điểm phù hợp với phạm vi của đề tài:

- **Ưu điểm:** Linh hoạt, có hệ sinh thái lớn và phù hợp với các tác vụ vào/ra.
- **Hạn chế:** Vòng lặp sự kiện không phù hợp với công việc tính toán nặng kéo dài trên cùng tiến trình, vì một tác vụ đồng bộ lớn có thể làm chậm các yêu cầu khác.
- **Cách áp dụng trong RabbitCV:** Hệ thống chủ yếu thực hiện truy vấn dữ liệu và gọi dịch vụ bên ngoài; các tác vụ nặng như suy luận mô hình được giao cho nhà cung cấp AI, nên mô hình của Node.js phù hợp với phạm vi đề tài.

3.2.2 Socket.IO

Socket.IO là một thư viện dành cho các ứng dụng web, mobile để phát triển các ứng dụng realtime (thời gian thực). Nó cung cấp một giao diện đơn giản để giao tiếp theo thời gian thực giữa trình duyệt và máy chủ [11]. Socket.IO sử dụng rất nhiều các công nghệ realtime như: WebSocket, Flash Socket, IFrame, JSONP polling,... Nó sẽ tự động chuyển sang WebSocket nếu có thể nên việc sử dụng Socket.IO trên trình duyệt cũng là đang sử dụng WebSocket vậy.

RabbitCV sử dụng Socket.IO cho chức năng trò chuyện thời gian thực giữa ứng viên và doanh nghiệp. Sau khi xác thực kết nối, mỗi socket được đưa vào phòng mang định danh người

dùng. Khi có tin nhắn mới, hệ thống lưu tin nhắn vào cơ sở dữ liệu trước, sau đó phát sự kiện đến phòng của người nhận.

Socket.IO trong RabbitCV không quản lý thông báo tuyển dụng. Thông báo được lấy qua REST API bằng fetch và được TanStack Query quản lý, làm mới theo chu kỳ. Socket.IO chỉ đóng vai trò thông báo nhanh về tin nhắn mới; lịch sử trò chuyện vẫn được lấy từ cơ sở dữ liệu qua API.

Những ưu điểm nổi bật của Socket.IO:

- **Tự động chuyển đổi giao thức thông minh:** Socket.IO có thể lựa chọn sử dụng Web-Socket, WebTransport hay HTTP long-polling,... khác mà không làm gián đoạn trải nghiệm người dùng.
- **Tự động phát hiện mất kết nối và kết nối lại:** Socket.IO sử dụng cơ chế heartbeat để kiểm tra trạng thái kết nối. Khi kết nối bị gián đoạn, máy khách tự thử kết nối lại với khoảng chờ tăng dần, hạn chế việc gửi quá nhiều yêu cầu cùng lúc.
- **Khả năng mở rộng nhiều máy chủ:** Khi hệ thống phát triển, Socket.IO có thể kết hợp các adapter như Redis Adapter để chia sẻ và đồng bộ các sự kiện giữa các server node một cách dễ dàng. Điều này rất hữu ích cho các ứng dụng web, mobile cần real-time như chat, game,....
- **Hệ sinh thái và hỗ trợ đa nền tảng:** Socket.IO cung cấp một hệ sinh thái phong phú với nhiều thư viện và công cụ hỗ trợ phát triển ứng dụng real-time. Nó có các thư viện client cho web, mobile (iOS, Android) và các framework phổ biến khác như React, Vue, Angular,..... giúp việc tích hợp vào ứng dụng trở nên dễ dàng và nhanh chóng.

3.3 Phát triển cơ sở dữ liệu

3.3.1 MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL hướng tài liệu [4]. Thay vì tổ chức dữ liệu thành bảng và hàng như cơ sở dữ liệu quan hệ, MongoDB lưu mỗi bản ghi dưới dạng một tài liệu BSON. BSON có cấu trúc gần với đối tượng JSON nhưng hỗ trợ thêm các kiểu dữ liệu phục vụ lưu trữ, chẳng hạn ngày giờ, dữ liệu nhị phân và mã định danh ObjectId. Các tài liệu được tập hợp trong collection, còn nhiều collection tạo thành một cơ sở dữ liệu. Cách biểu diễn này phù hợp với ứng dụng JavaScript vì dữ liệu có thể được chuyển đổi giữa đối tượng ở backend và tài liệu trong cơ sở dữ liệu mà không phải ánh xạ qua cấu trúc bảng phức tạp.

MongoDB cung cấp mô hình dữ liệu linh hoạt: các tài liệu trong cùng một collection không bắt buộc phải có toàn bộ trường giống nhau. Tuy nhiên, tính linh hoạt không đồng nghĩa với việc dữ liệu không cần thiết kế. Các tài liệu cùng loại vẫn cần duy trì cấu trúc hợp lý để truy vấn, kiểm tra và mở rộng ổn định. MongoDB cho phép nhúng đối tượng hoặc mảng vào một tài liệu, đồng thời cũng cho phép lưu mã định danh để tham chiếu đến tài liệu thuộc collection khác. Hai cách này giúp hệ thống lựa chọn cấu trúc theo vòng đời và cách sử dụng dữ liệu thay vì buộc mọi quan hệ phải được biểu diễn theo một hình thức duy nhất.

Những đặc điểm nổi bật của MongoDB gồm:

- **Mô hình tài liệu gần với dữ liệu ứng dụng:** Một tài liệu có thể chứa chuỗi, số, ngày giờ, đối tượng con và mảng. Dữ liệu nghiệp vụ vì vậy có thể được biểu diễn gần với cấu trúc mà Node.js tiếp nhận và trả về cho giao diện.
- **Hỗ trợ dữ liệu lồng nhau:** Những thông tin thường được đọc và cập nhật cùng nhau có thể được đặt trong cùng một tài liệu. Cách tổ chức này giảm nhu cầu thực hiện nhiều truy vấn chỉ để dựng lại một đối tượng nghiệp vụ hoàn chỉnh.

- **Truy vấn và tổng hợp dữ liệu:** MongoDB cung cấp các thao tác tạo, đọc, cập nhật và xóa, đồng thời hỗ trợ lọc, sắp xếp, phân trang và aggregation pipeline để xử lý những yêu cầu tổng hợp phức tạp hơn.
- **Hệ thống chỉ mục:** Chỉ mục có thể được tạo trên một trường hoặc tổ hợp nhiều trường để tăng tốc các truy vấn thường xuyên. Chỉ mục duy nhất còn có thể thực thi những ràng buộc không cho phép dữ liệu trùng lặp.
- **Khả năng mở rộng:** MongoDB hỗ trợ sao chép dữ liệu để tăng tính sẵn sàng và phân mảnh dữ liệu để mở rộng theo chiều ngang. Đây là khả năng của nền tảng khi hệ thống phát triển; RabbitCV hiện sử dụng MongoDB trong phạm vi phù hợp với quy mô đồ án và không khẳng định đã triển khai các cơ chế mở rộng này.

Trong RabbitCV, MongoDB lưu tài khoản người dùng, CV, tin tuyển dụng, hồ sơ ứng tuyển, tin nhắn, thông báo, trạng thái trò chuyện và thống kê sử dụng AI. Một CV có thể chứa nhiều nhóm thông tin như học vấn, kinh nghiệm, kỹ năng, dự án, chứng chỉ và ngôn ngữ; mỗi nhóm lại có số lượng phần tử khác nhau giữa các người dùng. Mô hình tài liệu giúp các nhóm dữ liệu này được biểu diễn tự nhiên dưới dạng đối tượng và mảng, đồng thời vẫn có thể liên kết CV với chủ sở hữu hoặc liên kết hồ sơ ứng tuyển với công việc bằng mã định danh.

3.3.2 Mongoose

Mongoose là thư viện mô hình hóa dữ liệu đối tượng dành cho Node.js và MongoDB [1]. Thư viện tạo một lớp làm việc có cấu trúc giữa mã nguồn backend và cơ sở dữ liệu, cung cấp cách khai báo hình dạng tài liệu, kiểu dữ liệu, quy tắc kiểm tra và các thao tác truy vấn. Mongoose không phải là cơ sở dữ liệu và không thay thế MongoDB; dữ liệu cuối cùng vẫn được lưu, truy vấn và lập chỉ mục bởi MongoDB. Ứng dụng có thể sử dụng MongoDB Driver trực tiếp, nhưng Mongoose giúp giảm phần mã lặp khi nhiều nhóm dữ liệu cần tuân theo các quy tắc thống nhất.

Bốn khái niệm chính của Mongoose gồm:

- **Schema:** Mô tả các trường dự kiến của một loại tài liệu, kiểu dữ liệu, trường bắt buộc, giá trị mặc định, tập giá trị hợp lệ và chỉ mục. Schema cũng có thể chứa phương thức và các bước xử lý được thực hiện trong vòng đời tài liệu.
- **Model:** Được tạo từ Schema và đại diện cho một collection. Model cung cấp những thao tác như tạo tài liệu, tìm kiếm, cập nhật, xóa và tổng hợp dữ liệu.
- **Document:** Là một bản ghi cụ thể được tạo hoặc đọc thông qua Model. Document có thể được kiểm tra, thay đổi và lưu trở lại MongoDB.
- **Query:** Biểu diễn điều kiện và tùy chọn của một thao tác truy vấn. Các phương thức có thể được nối tiếp để lọc trường, sắp xếp, giới hạn kết quả, phân trang hoặc lấy thêm tài liệu liên quan.

Mongoose hỗ trợ chuyển đổi dữ liệu về kiểu đã khai báo trước khi thực hiện thao tác. Nếu một giá trị không thể chuyển đổi hợp lệ, thư viện phát sinh lỗi thay vì âm thầm lưu dữ liệu sai kiểu. Các validator có sẵn như `required`, `min`, `max` và `enum` giúp kiểm tra những điều kiện cơ bản; dự án cũng có thể định nghĩa validator riêng cho quy tắc đặc thù. Giá trị mặc định và tùy chọn `timestamps` giúp các Model sử dụng thống nhất trạng thái ban đầu, thời điểm tạo và thời điểm cập nhật.

Những ưu điểm nổi bật của Mongoose gồm:

- **Tổ chức cấu trúc dữ liệu tập trung:** Quy tắc của một nhóm dữ liệu được đặt trong Schema tương ứng, giúp nhà phát triển dễ xác định trường nào được lưu và điều kiện nào phải được đáp ứng.

- **Giảm mã truy vấn thủ công:** Model cung cấp giao diện nhất quán cho các thao tác cơ sở dữ liệu, giúp mã xử lý nghiệp vụ dễ đọc hơn so với việc tự xây dựng mọi lệnh ở tầng Driver.
- **Kiểm tra và chuyển đổi dữ liệu:** Dữ liệu được kiểm tra và ép kiểu trước khi ghi, qua đó phát hiện sớm nhiều lỗi về trường bắt buộc, kiểu dữ liệu hoặc trạng thái không hợp lệ.
- **Hỗ trợ quan hệ tham chiếu:** Kiểu ObjectId, thuộc tính ref và cơ chế populate hỗ trợ lấy dữ liệu liên quan từ collection khác khi nghiệp vụ cần sử dụng.
- **Hỗ trợ bảo trì và mở rộng:** Middleware, phương thức dùng chung, trường ảo, chỉ mục và tùy chọn chọn trường cho phép đặt những hành vi lặp lại gần với mô hình dữ liệu thay vì phân tán ở nhiều vị trí xử lý.

Trong RabbitCV, Mongoose được sử dụng để quy định các trường bắt buộc, vai trò và trạng thái hợp lệ, gán giá trị mặc định, tự động ghi thời gian và mô tả quan hệ giữa các tài liệu. Những trường nhạy cảm như mật khẩu và mã PIN được cấu hình không xuất hiện trong truy vấn thông thường. Các Model cũng khai báo chỉ mục phục vụ đăng nhập, tra cứu dữ liệu theo người sở hữu, theo dõi thông báo và ngăn hồ sơ ứng tuyển trùng. Tuy nhiên, validation của Mongoose chỉ bảo vệ các thao tác đi qua Mongoose; các ràng buộc quan trọng vẫn cần chỉ mục hoặc cơ chế kiểm tra ở MongoDB và không thay thế kiểm tra nghiệp vụ tại backend.

3.3.3 Sự kết hợp giữa MongoDB và Mongoose

MongoDB và Mongoose đảm nhiệm hai tầng khác nhau nhưng bổ sung cho nhau. MongoDB chịu trách nhiệm lưu trữ bền vững, thực thi truy vấn, quản lý chỉ mục và trả kết quả. Mongoose nằm trong ứng dụng Node.js, tiếp nhận dữ liệu từ nghiệp vụ, áp dụng Schema, chuyển đổi kiểu, kiểm tra giá trị rồi sử dụng MongoDB Driver để gửi thao tác đến cơ sở dữ liệu. Kết quả từ MongoDB được Mongoose chuyển thành Document hoặc đối tượng để backend tiếp tục xử lý và trả về giao diện [4, 1].

Một thao tác ghi dữ liệu thông thường được thực hiện theo trình tự sau:

1. Express tiếp nhận yêu cầu và backend kiểm tra phiên đăng nhập, quyền truy cập cùng các điều kiện nghiệp vụ.
2. Dữ liệu hợp lệ về mặt nghiệp vụ được chuyển cho Model Mongoose tương ứng.
3. Mongoose chuyển đổi kiểu, áp dụng giá trị mặc định và thực hiện validation theo Schema.
4. MongoDB nhận thao tác ghi, thực thi chỉ mục và các ràng buộc ở tầng cơ sở dữ liệu trước khi lưu tài liệu.
5. Kết quả hoặc lỗi được trả về qua Mongoose để backend tạo phản hồi phù hợp cho giao diện.

Sự kết hợp này mang lại các lợi ích chính sau:

- **Kết hợp tính linh hoạt với tính nhất quán:** MongoDB cho phép cấu trúc tài liệu thích ứng với dữ liệu nghiệp vụ, trong khi Mongoose giúp các tài liệu do RabbitCV tạo ra tuân theo hình dạng và quy tắc dự kiến.
- **Phù hợp với hệ sinh thái JavaScript:** Dữ liệu được biểu diễn dưới dạng đối tượng từ giao diện, qua Express và Mongoose đến MongoDB, giúp giảm số bước chuyển đổi giữa các mô hình dữ liệu khác nhau.
- **Giảm mã lặp ở backend:** Kiểu dữ liệu, giá trị mặc định, validation, thời gian và thao tác truy vấn được cung cấp qua Schema và Model thay vì được xây dựng lại trong từng đường dẫn API.
- **Hỗ trợ cả dữ liệu lỏng lẻo và dữ liệu liên kết:** Các thành phần thuộc cùng một CV có thể được nhúng trong một tài liệu, còn người dùng, công việc, CV và hồ sơ ứng tuyển có thể liên kết bằng ObjectId khi chúng có vòng đời riêng.

- **Tạo nhiều lớp bảo vệ dữ liệu:** Backend kiểm tra nghiệp vụ và quyền sở hữu, Mongoose kiểm tra cấu trúc, còn MongoDB thực thi chỉ mục và ràng buộc dữ liệu. Mỗi lớp giải quyết một loại lỗi khác nhau và không thay thế lẫn nhau.
- **Tăng khả năng bảo trì:** Khi bổ sung trường hoặc trạng thái mới, thay đổi có thể được quản lý tập trung tại Model liên quan, sau đó các chức năng sử dụng chung cùng một cách biểu diễn dữ liệu.

Trong RabbitCV, lợi ích của sự kết hợp thể hiện rõ ở dữ liệu CV và hồ sơ ứng tuyển. Nội dung CV được biểu diễn bằng các đối tượng và mảng lồng nhau, trong khi quyền sở hữu và quan hệ ứng tuyển được biểu diễn bằng mã định danh. Mongoose giúp backend đọc và ghi những cấu trúc này bằng Model thống nhất; MongoDB cung cấp nơi lưu trữ, truy vấn và chỉ mục để dữ liệu vẫn được duy trì giữa các phiên sử dụng. Nhờ phân chia trách nhiệm rõ ràng, hệ thống vừa giữ được sự linh hoạt cần thiết cho nội dung CV, vừa kiểm soát được những trường bắt buộc và ràng buộc quan trọng của quy trình tuyển dụng.

3.3.4 Dữ liệu lồng nhau và quan hệ tham chiếu

Hai cách tổ chức chính trong MongoDB là nhúng dữ liệu vào cùng tài liệu và lưu quan hệ bằng tham chiếu. Dữ liệu được nhúng phù hợp khi các phần thường được đọc cùng nhau, có cùng chủ sở hữu và không cần cập nhật độc lập. Trong RabbitCV, nhiều mục thành phần của CV được tổ chức dưới dạng mảng lồng nhau vì chúng chỉ có ý nghĩa trong hồ sơ tương ứng và thường được hiển thị cùng lúc.

Tham chiếu phù hợp khi tài liệu có vòng đời riêng, được nhiều nghiệp vụ sử dụng hoặc có thể tăng trưởng độc lập. Người dùng, tin tuyển dụng, CV và đơn ứng tuyển được liên kết bằng mã định danh để backend có thể kiểm tra chủ sở hữu và truy vấn theo từng phạm vi. Mongoose hỗ trợ khai báo tham chiếu và lấy thêm dữ liệu liên quan khi cần, nhưng dự án vẫn kiểm soát các trường được chọn để tránh trả về thông tin nhạy cảm hoặc dữ liệu không cần thiết.

Đối với lịch sử ứng tuyển, RabbitCV lưu thêm bản chụp thông tin quan trọng của công việc tại thời điểm nộp hồ sơ. Bản chụp giúp đơn ứng tuyển vẫn còn ngữ cảnh về vị trí, doanh nghiệp và nội dung cơ bản ngay cả khi tin tuyển dụng gốc đã bị xóa hoặc thay đổi. Đây là sự kết hợp giữa tham chiếu và dữ liệu bất biến theo thời điểm, phù hợp với yêu cầu lưu lịch sử nghiệp vụ.

Việc lựa chọn nhúng hay tham chiếu ảnh hưởng đến số lượng truy vấn, kích thước tài liệu và độ phức tạp khi cập nhật. Nhúng giúp đọc nhanh theo cụm nhưng có thể làm tài liệu lớn; tham chiếu giảm trùng lặp nhưng cần thêm truy vấn hoặc phép nối ở tầng ứng dụng. RabbitCV lựa chọn theo vòng đời và tần suất sử dụng dữ liệu thay vì áp dụng một cách duy nhất cho toàn hệ thống.

3.3.5 Chỉ mục, ràng buộc và tối ưu dữ liệu

Chỉ mục giúp MongoDB tìm tài liệu theo trường hoặc tổ hợp trường mà không phải quét toàn bộ tập dữ liệu. RabbitCV tạo chỉ mục cho những thuộc tính thường dùng để đăng nhập, lọc hoặc liên kết. Chỉ mục duy nhất được dùng cho dữ liệu không được phép trùng như tên đăng nhập, thư điện tử và tổ hợp định danh công việc–ứng viên trong đơn ứng tuyển. Ràng buộc ở cơ sở dữ liệu ngăn hai yêu cầu đồng thời tạo bản ghi trùng ngay cả khi cả hai đã vượt qua bước kiểm tra ở mã ứng dụng.

Ngoài chỉ mục, dự án dùng giá trị liệt kê, trường bắt buộc, giá trị mặc định và quy tắc kiểm tra của Mongoose để duy trì trạng thái hợp lệ. Quyền sở hữu được thể hiện bằng mã người dùng hoặc doanh nghiệp liên quan. Trạng thái đọc và ẩn của trò chuyện được lưu theo từng người dùng, nhờ đó thao tác của một bên không xóa hoặc thay đổi lịch sử chung của bên còn lại.

Tối ưu dữ liệu không chỉ nằm ở truy vấn cơ sở dữ liệu mà còn ở nội dung phản hồi. CV PDF tải lên có thể chứa chuỗi mã hóa dung lượng lớn; khi trả danh sách CV, backend loại phần dữ liệu PDF và chỉ gửi thông tin mô tả cần cho danh sách. Nội dung đầy đủ chỉ được lấy khi người dùng mở một CV cụ thể. Cách chiếu trường này giảm lượng dữ liệu truyền, thời gian phân tích JSON và bộ nhớ dùng ở frontend.

Chỉ mục làm tăng tốc độ đọc nhưng tiêu tốn dung lượng và làm mỗi thao tác ghi phải cập nhật thêm cấu trúc chỉ mục. Vì vậy, chỉ mục cần dựa trên truy vấn thực tế thay vì tạo cho mọi trường. Tương tự, ràng buộc model và chỉ mục không thay thế kiểm tra nghiệp vụ; chúng hoạt động như lớp bảo vệ bổ sung để giữ dữ liệu nhất quán khi có lỗi hoặc yêu cầu đồng thời.

3.4 Phát triển trí tuệ nhân tạo

3.4.1 Mô hình ngôn ngữ lớn, Groq và OpenAI SDK

Mô hình ngôn ngữ lớn có khả năng tạo, biến đổi và phân tích văn bản dựa trên chỉ dẫn cùng ngữ cảnh đầu vào. RabbitCV sử dụng khả năng này để hỗ trợ viết hoặc cải thiện phần giới thiệu, rà soát nội dung CV, gợi ý bổ sung, đánh giá mức độ phù hợp giữa CV và công việc, đồng thời tạo hội thoại luyện phỏng vấn. Mô hình giúp xử lý nội dung linh hoạt hơn các tập luật cố định, đặc biệt khi cách diễn đạt và kinh nghiệm của mỗi người dùng khác nhau.

Backend sử dụng OpenAI SDK để tạo yêu cầu theo giao diện lập trình phổ biến, nhưng cấu hình địa chỉ API tương thích OpenAI của Groq. Cách tích hợp này giúp mã gọi mô hình có cấu trúc rõ ràng và có thể cấu hình nhà cung cấp, mô hình, thời gian chờ cũng như tham số sinh nội dung ở phía máy chủ. Groq hỗ trợ thực thi mô hình với độ trễ thấp, phù hợp với các chức năng cần phản hồi trong lúc người dùng đang chỉnh sửa CV hoặc luyện phỏng vấn.

Khóa truy cập được lưu trong biến môi trường của backend và không được đưa xuống trình duyệt. Frontend chỉ gửi yêu cầu đến API của RabbitCV; backend chịu trách nhiệm xác thực người dùng, kiểm soát hạn mức, rút gọn dữ liệu và gọi nhà cung cấp. Cách đặt lớp trung gian này bảo vệ khóa, thống nhất chính sách và cho phép thay đổi cấu hình mà không phải sửa mã giao diện.

Mô hình ngôn ngữ được sử dụng với các điểm cần lưu ý sau:

- **Ưu điểm:** Có khả năng thích ứng và tạo gợi ý theo ngữ cảnh.
- **Hạn chế:** Kết quả có thể không chính xác, suy diễn quá mức hoặc không phù hợp với dữ liệu thật của người dùng.
- **Nguyên tắc sử dụng:** RabbitCV xem đầu ra AI là đề xuất cần được người dùng xem xét; mô hình không tự ghi đè CV và không thay người dùng đưa ra quyết định cuối cùng.

3.4.2 JSON Schema và đầu ra có cấu trúc

JSON Schema mô tả hình dạng của tài liệu JSON, bao gồm tên trường, kiểu dữ liệu, trường bắt buộc, giá trị cho phép và cấu trúc lồng nhau [3]. RabbitCV định nghĩa schema riêng cho từng chức năng AI thay vì yêu cầu mô hình trả một đoạn văn bản tự do. Ví dụ, kết quả rà soát cần tách nhận xét và đề xuất; kết quả so khớp cần có điểm và các nhóm phân tích; hội thoại phỏng vấn cần phân biệt câu hỏi, phản hồi và bước tiếp theo.

Yêu cầu đầu ra có cấu trúc giúp backend nhận dữ liệu gần với hình dạng mà giao diện cần sử dụng. Chế độ structured output của nhà cung cấp được dùng để buộc kết quả tuân theo schema đã chọn [2]. Sau khi nhận phản hồi, backend vẫn phân tích JSON, kiểm tra trường và chuẩn hóa

giá trị trước khi trả cho frontend. Cách làm này giảm lỗi do tách văn bản thủ công và giúp thành phần giao diện không phải dự đoán cách mô hình trình bày câu trả lời.

Schema cũng tạo hợp đồng rõ ràng giữa mô-đun AI và phần còn lại của hệ thống. Khi muốn thay đổi giao diện hoặc thêm trường kết quả, nhà phát triển có thể cập nhật schema và bộ xử lý tương ứng. Việc kiểm thử trở nên cụ thể hơn vì có thể xác nhận kiểu, giới hạn và trường bắt buộc thay vì chỉ so sánh một chuỗi văn bản dài.

Tuy nhiên, đúng cấu trúc không đồng nghĩa đúng nội dung. Một điểm số có thể nằm trong giới hạn cho phép nhưng lập luận vẫn chưa hợp lý; một gợi ý có thể đúng ngữ pháp nhưng không phản ánh kinh nghiệm thật. Do đó, kiểm tra schema chỉ giải quyết tính máy đọc được, còn độ tin cậy ngữ nghĩa cần được hỗ trợ bằng prompt rõ ràng, giới hạn dữ liệu, thông báo cho người dùng và quyền xác nhận cuối cùng.

3.4.3 Kiểm soát đầu vào, hạn mức và thống kê sử dụng

Dữ liệu CV và công việc có thể dài, chứa thông tin liên hệ hoặc nội dung cố gắng thay đổi chỉ dẫn của hệ thống. Trước khi gửi đến mô hình, RabbitCV chuẩn hóa văn bản, giới hạn độ dài, giới hạn số phần tử và loại hoặc thay thế thông tin liên hệ trực tiếp khi không cần thiết. Prompt hệ thống phân biệt dữ liệu người dùng với chỉ dẫn, đồng thời yêu cầu mô hình chỉ thực hiện đúng nhiệm vụ đã định nghĩa. Các bước này làm giảm lượng dữ liệu gửi ra ngoài và hạn chế ảnh hưởng của nội dung không đáng tin cậy.

Mỗi đường dẫn AI được bảo vệ bằng xác thực, giới hạn tốc độ và thời gian chờ. Hệ thống còn áp dụng hạn mức theo ngày cho từng tài khoản để kiểm soát chi phí và ngăn một người dùng chiếm toàn bộ năng lực dịch vụ. Khi nhà cung cấp phản hồi chậm hoặc không khả dụng, yêu cầu được kết thúc có kiểm soát để tiến trình máy chủ không chờ vô hạn và giao diện có thể hướng dẫn người dùng thử lại.

RabbitCV lưu thống kê kỹ thuật như loại chức năng, số lượt gọi, lượng token, độ trễ và trạng thái lỗi. Dữ liệu đầu vào được nhận diện bằng giá trị băm khi cần đối chiếu thay vì lưu lại toàn bộ nội dung CV trong bản ghi thống kê. Cách thu thập này hỗ trợ đánh giá mức sử dụng, phát hiện bất thường và ước lượng chi phí nhưng hạn chế sao chép dữ liệu cá nhân không cần thiết.

Những biện pháp trên phù hợp với cách quản lý rủi ro AI theo hướng minh bạch, có thể kiểm tra và lấy con người làm trung tâm [12]. Đây mới là phần giới thiệu nền tảng công nghệ; kiến trúc, năm luồng AI, cấu trúc prompt và cách xử lý kết quả được trình bày chi tiết tại Chương 6.

3.5 Phát triển tạo và xuất tệp PDF

3.5.1 Bố cục A4 và react-to-print

CV có cấu trúc được RabbitCV kết xuất từ dữ liệu thành các thành phần HTML và CSS. Vùng nội dung chính được thiết kế theo khổ A4 với kích thước 210×297 mm, lề và kiểu chữ phù hợp với tài liệu in. Bố cục trên màn hình có thể được thu phóng để vừa cửa sổ, trong khi quy tắc dành cho in giữ kích thước vật lý và loại bỏ những thành phần điều khiển không thuộc nội dung CV.

Thư viện `react-to-print` kết nối một vùng thành phần React với chức năng in của trình duyệt. RabbitCV sử dụng hook dùng chung để chọn nội dung cần in, thiết lập tên tài liệu, áp dụng quy tắc trang và xử lý lỗi. Khi mở hộp thoại in, người dùng có thể chọn máy in hoặc chức năng lưu thành PDF có sẵn trong trình duyệt. Cách tiếp cận này tận dụng công cụ bố cục của trình duyệt và không yêu cầu backend dựng lại toàn bộ CV bằng một thư viện PDF riêng.

Ưu điểm của phương pháp HTML/CSS là mẫu CV có thể dùng chung cho xem trước và xuất, thay đổi dữ liệu được phản ánh ngay và nhà phát triển có thể xây dựng mẫu bằng cùng công nghệ với giao diện. Việc bổ sung mẫu mới cũng chủ yếu là tạo thành phần trình bày thay vì mô tả tọa độ từng dòng trong tệp PDF.

Giới hạn là kết quả in có thể khác nhẹ giữa trình duyệt, phong chữ và thiết lập lề của người dùng. Nội dung dài còn có thể phát sinh ngắt trang không mong muốn. Vì vậy, các mẫu cần quy tắc CSS dành riêng cho in, kiểm tra với dữ liệu ngắn và dài, đồng thời hướng dẫn người dùng chọn khổ giấy, tỷ lệ và lề thích hợp khi lưu PDF.

3.5.2 PDF.js và hiển thị PDF tải lên

Ngoài CV được tạo trong hệ thống, RabbitCV cho phép người dùng tải lên CV PDF đã có. PDF.js là thư viện đọc và kết xuất tài liệu PDF trong trình duyệt [5]. Thư viện tải tài liệu, xác định số trang và vẽ từng trang lên vùng hiển thị. Phần xử lý chính có thể chạy qua worker để tránh chặn luồng giao diện khi giải mã nội dung PDF.

RabbitCV sử dụng PDF.js để cung cấp trải nghiệm xem trước thống nhất thay vì phụ thuộc hoàn toàn vào trình đọc PDF tích hợp của từng trình duyệt. Giao diện có thể kiểm soát tỷ lệ, thứ tự trang, trạng thái đang tải và lỗi tài liệu. Người dùng nhờ đó có thể xem lại CV đã tải lên ngay trong luồng quản lý hoặc ứng tuyển mà không phải mở một ứng dụng bên ngoài.

PDF.js có các đặc điểm chính như sau:

- **Ưu điểm:** Có khả năng hiển thị đa nền tảng và tích hợp sâu với giao diện web.
- **Hạn chế:** Tài liệu nhiều trang hoặc có đồ họa phức tạp tiêu tốn bộ nhớ và thời gian kết xuất; worker và tệp thư viện cũng làm tăng kích thước gói frontend.
- **Cách tối ưu trong RabbitCV:** Nhóm PDF được tách thành gói riêng và chỉ được tải khi người dùng mở chức năng liên quan.

PDF tải lên là tài liệu gần như hoàn chỉnh, khác với CV có cấu trúc mà hệ thống có thể chỉnh sửa từng mục. RabbitCV phân biệt hai loại để chọn đúng cách hiển thị, cập nhật và sử dụng. Sự phân biệt này cũng tránh áp dụng nhầm thao tác chỉnh sửa cấu trúc cho một tệp chỉ có thể xem hoặc thay thế.

3.5.3 Kiểm tra, lưu trữ và tối ưu dữ liệu PDF

Không nên tin cậy tệp chỉ dựa trên tên hoặc phần mở rộng do trình duyệt gửi lên. Backend RabbitCV kiểm tra định dạng data URL, kiểu MIME, phần mở rộng, phần đầu tệp sau giải mã và dung lượng thực. Tệp chỉ được chấp nhận khi các dấu hiệu đều phù hợp với PDF và kích thước nằm trong giới hạn của hệ thống. Kiểm tra nhiều lớp giúp loại bỏ dữ liệu sai định dạng sớm và giảm nguy cơ lưu một nội dung không phải PDF dưới tên tệp PDF.

CV cấu trúc và CV tải lên được đánh dấu bằng thuộc tính trạng thái để backend biết tài liệu được tạo từ dữ liệu biểu mẫu hay từ tệp. Với CV cấu trúc, dữ liệu các mục là nguồn chính và giao diện có thể kết xuất lại mẫu. Với CV tải lên, nội dung PDF là nguồn chính và người dùng thao tác trên tệp đã có. Cách phân loại này giúp các luồng xem, sửa, chọn CV khi ứng tuyển và xuất tài liệu hoạt động đúng theo loại hồ sơ.

Dữ liệu PDF được mã hóa có kích thước lớn hơn tệp nhị phân ban đầu. Nếu đưa trường này vào mọi phản hồi danh sách, giao diện phải tải lượng dữ liệu lớn dù chỉ cần tên, ngày cập nhật và loại CV. RabbitCV dùng phép chiếu để loại dữ liệu PDF khỏi truy vấn danh sách; nội dung đầy đủ chỉ được trả khi mở một hồ sơ cụ thể. Biện pháp này giảm băng thông, thời gian xử lý JSON và mức sử dụng bộ nhớ ở cả máy chủ lẫn trình duyệt.

Cách lưu dữ liệu mã hóa trong cơ sở dữ liệu phù hợp với quy mô và phạm vi hiện tại vì đơn giản hóa quá trình triển khai. Tuy nhiên, nếu số lượng hoặc dung lượng tệp tăng mạnh, hệ thống nên cân nhắc kho lưu trữ đối tượng và chỉ giữ đường dẫn cùng siêu dữ liệu trong MongoDB. Đây là giới hạn mở rộng cần được theo dõi, không làm thay đổi quy trình kiểm tra và phân quyền tải tệp.

Chương 4

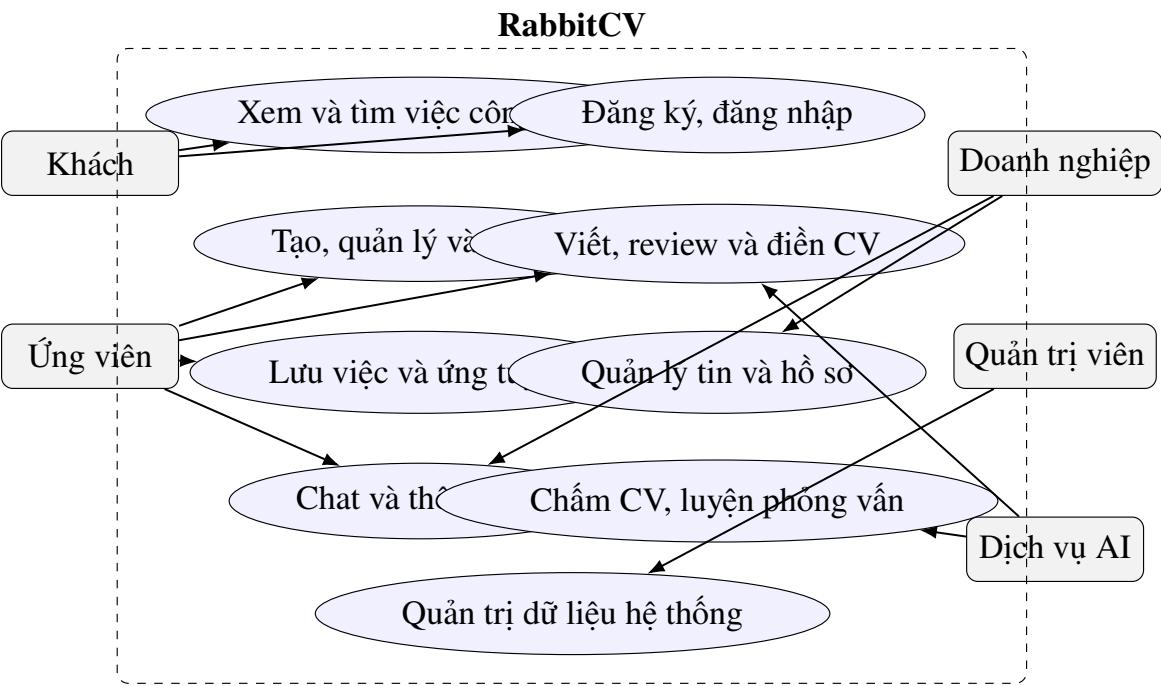
Phân tích và thiết kế hệ thống

4.1 Phân tích tác nhân

RabbitCV có bốn tác nhân chính:

- **Khách:** chỉ truy cập tài nguyên công khai.
- **Ứng viên:** là tác nhân trung tâm của đề tài vì trực tiếp tạo CV, nhận gợi ý AI và dùng CV để ứng tuyển.
- **Doanh nghiệp:** tạo tin, tiếp nhận hồ sơ và trao đổi với ứng viên.
- **Quản trị viên:** giám sát dữ liệu và thực hiện các thao tác đặc quyền trong phạm vi backend đã triển khai.

Tác nhân ngoài hệ thống là dịch vụ AI Groq. Dịch vụ này không truy cập trực tiếp cơ sở dữ liệu; mọi yêu cầu đều đi qua backend để xác thực, rút gọn dữ liệu, áp dụng quota và yêu cầu đầu ra theo schema.



Hình 4.1: Bản đồ tác nhân và nhóm ca sử dụng chính

4.2 Yêu cầu chức năng

Các yêu cầu được gán mã để liên kết với module triển khai và kiểm thử.

Mã	Nhóm	Yêu cầu
FR-CV-01	CV	Ứng viên tạo, đọc, cập nhật, xóa Resume và chọn template.
FR-CV-02	CV	Hệ thống preview CV theo A4, hỗ trợ in/tải PDF và tiếp tục chỉnh sửa dữ liệu cấu trúc.
FR-CV-03	PDF	Trong luồng ứng tuyển, ứng viên có thể tải CV PDF hợp lệ; doanh nghiệp xem từng trang bằng PDF.js.
FR-AI-01	AI viết CV	AI viết mới, cải thiện hoặc rút gọn phần tiểu sử; người dùng xem kết quả và quyết định áp dụng.
FR-AI-02	AI review/fill	AI review CV theo nhóm tiêu chí và gợi ý điền mục còn thiếu mà không tự khẳng định sự kiện cá nhân chưa có.
FR-AI-03	AI mở rộng	Hệ thống chấm mức độ phù hợp CV–Job và luyện phỏng vấn theo từng lượt.
FR-JOB-01	Việc làm	Người dùng tìm, lọc, xem, lưu và bỏ lưu tin tuyển dụng.
FR-APP-01	Ứng tuyển	Ứng viên nộp CV đã lưu hoặc PDF, không được nộp trùng và không nộp vào tin không khả dụng.
FR-APP-02	Lịch sử	Application giữ snapshot thông tin Job và trạng thái xử lý hồ sơ.
FR-COM-01	Doanh nghiệp	Doanh nghiệp tạo/sửa/xóa tin, quản lý nhận hồ sơ, xem ứng viên và cập nhật trạng thái.
FR-MSG-01	Giao tiếp	Hai bên gửi tin nhắn, theo dõi trạng thái đọc, ẩn room theo từng người và nhận thông báo nghiệp vụ.
FR-ADM-01	Quản trị	Admin xem thống kê và quản lý người dùng, doanh nghiệp, tin trong phạm vi API có sẵn.

Bảng 4.1: Danh sách yêu cầu chức năng

4.3 Yêu cầu phi chức năng

Mã	Nhóm	Tiêu chí kiểm chứng
NFR-SEC-01	Xác thực	Mọi API riêng tư và toàn bộ route AI yêu cầu JWT hợp lệ; thao tác đặc quyền phải kiểm tra vai trò ở backend. API chung, đăng nhập, đăng ký và AI có giới hạn tần suất riêng.
NFR-SEC-02	Quyền dữ liệu	Resume chỉ được trả cho chủ sở hữu, admin hoặc doanh nghiệp có quan hệ nghiệp vụ phù hợp.
NFR-PDF-01	Tài liệu	CV web dùng kích thước A4; PDF upload phải có header hợp lệ và kích thước giải mã không quá 5 MB.
NFR-AI-01	Kiểm soát AI	Rate limit 30 request/15 phút; quota ngày và timeout có thể cấu hình, mặc định lần lượt là 100 request và 20 giây.
NFR-AI-02	Hợp đồng dữ liệu	Request AI loại thông tin liên hệ trực tiếp; response phải khớp JSON Schema và được làm sạch trước khi trả frontend.
NFR-DATA-01	Toàn vẹn	Unique index ngăn ứng tuyển trùng; snapshot giữ lịch sử; thao tác ẩn/đọc chat không xóa dữ liệu của bên còn lại.
NFR-TEST-01	Khả năng tái lập	Quy trình npm run check phải chạy ESLint, 41 test hiện có và Vite build; kết quả phải gắn với ngày, phiên bản runtime và trạng thái working tree.
NFR-UX-01	Khả dụng	Kết quả AI chỉ là gợi ý, có trạng thái chờ/lỗi và không tự ghi đè nội dung khi người dùng chưa xác nhận.

Bảng 4.2: Yêu cầu phi chức năng có tiêu chí kiểm chứng

4.4 Ma trận phân quyền

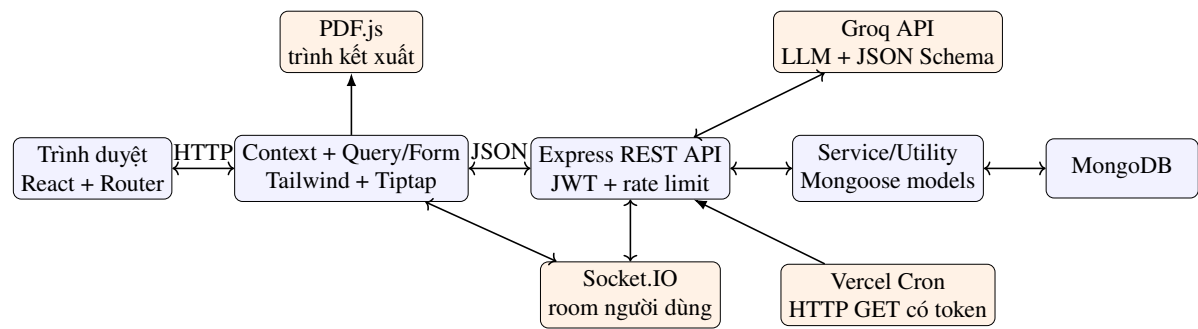
Chức năng	Khách	Ứng viên	Doanh nghiệp	Admin
Xem danh sách/chi tiết việc công khai	Có	Có	Có	Có
Tạo, sửa và dùng AI cho CV cá nhân	Không	Có	Không	Giới hạn
Lưu việc và nộp hồ sơ	Không	Có	Không	Không
Tạo và quản lý tin tuyển dụng	Không	Không	Tin sở hữu	Đặc quyền
Xem CV ứng viên	Không	CV sở hữu	Hồ sơ liên quan	Đặc quyền
Cập nhật trạng thái Application	Không	Không	Hồ sơ thuộc tin sở hữu	Đặc quyền
Nhắn tin và đọc thông báo	Không	Có	Có	Theo phạm vi
Quản lý người dùng/doanh nghiệp	Không	Không	Không	Có

Bảng 4.3: Ma trận quyền theo tác nhân

Ma trận trên mô tả chính sách mong muốn ở mức nghiệp vụ. Frontend có thể ẩn hoặc chuyển hướng giao diện, nhưng quyết định cuối cùng phải do middleware và truy vấn có điều kiện ở backend thực hiện. Qua đối chiếu mã nguồn hiện tại, nhóm API AI mới yêu cầu JWT chung và chưa kiểm tra riêng vai trò ứng viên; một số endpoint lưu việc và chat cũng mới dừng ở mức xác thực. Đây là khoảng cách giữa chính sách và hiện thực cần được kiểm thử, bổ sung trước khi triển khai thực tế.

4.5 Kiến trúc tổng thể

RabbitCV là ứng dụng client-server, không phải microservice. Frontend và backend có thể đóng gói/chạy riêng nhưng backend hiện tập trung phần lớn REST route trong `server/server.js`. Route AI, schema, service gọi nhà cung cấp, sanitizer và hàm xử lý hạn tin được tách thành các module riêng. Ở frontend, Context quản lý phiên và socket; TanStack Query quản lý một phần dữ liệu máy chủ; React Hook Form và Zod quản lý biểu mẫu xác thực.



Hình 4.2: Kiến trúc triển khai của RabbitCV

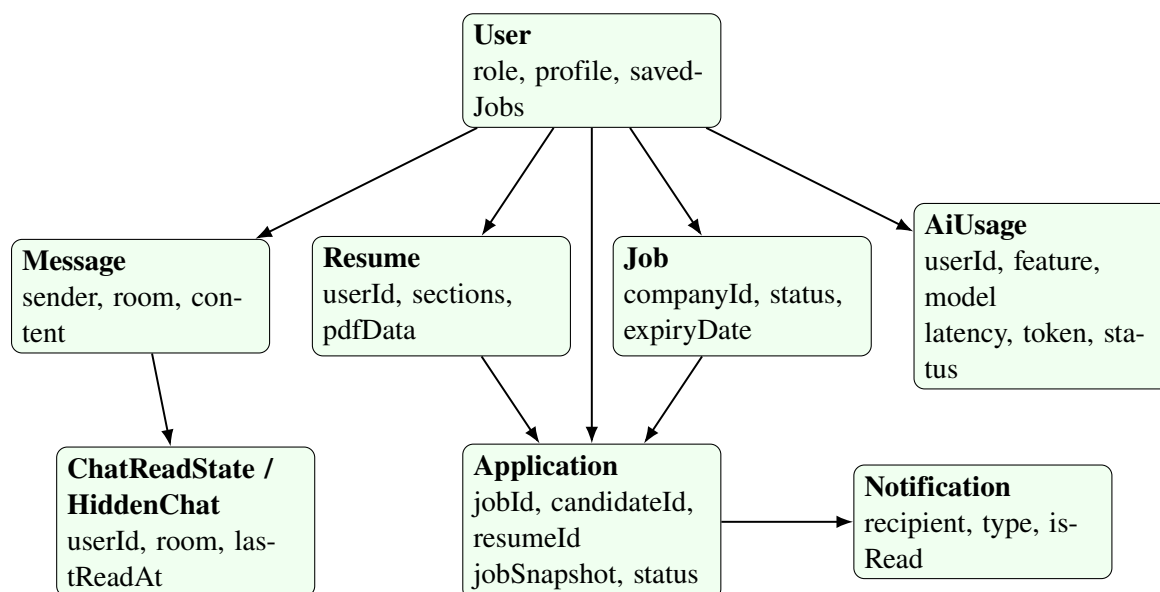
4.6 Thiết kế mô-đun

Mô-đun	Thành phần chính	Trách nhiệm
Phiên và vai trò	AuthContext, RoleAccess-Guard, authMiddleware.	Đăng ký, đăng nhập, khôi phục phiên, đăng xuất, chuyển hướng và kiểm tra quyền.
Dữ liệu giao diện	QueryClient, SavedJobsContext, useNotifications, form schema.	Cache dữ liệu máy chủ, polling, cập nhật lạc quan, khôi phục khi lỗi và kiểm tra biểu mẫu.
CV	ResumeForm, ResumePreview, Resume model.	Dữ liệu CV, template, preview A4, in PDF, CV upload và quyền sở hữu.
AI	aiRoutes, aiService, aiSchemas, sanitizer, AiUsage.	Chuẩn hóa input, quota, gọi model, kiểm tra schema, lọc output và thông kê.
Việc làm	Jobs, JobDetail, Job/Application models.	Tìm kiếm, trạng thái tin, lưu việc, ứng tuyển, snapshot và xử lý hồ sơ.
Giao tiếp	Message, SocketContext, NotificationBell.	Lưu message, phát event, trạng thái đọc/ẩn room và thông báo.
Doanh nghiệp	CompanyDashboard và API Job/Application.	Hồ sơ công ty, tin tuyển dụng, danh sách ứng viên và cập nhật trạng thái.
Quản trị	AdminLayout và nhóm API admin.	Thống kê, người dùng, doanh nghiệp và tin tuyển dụng.

Bảng 4.4: Phân rã mô-đun theo thành phần triển khai

4.7 Thiết kế dữ liệu

MongoDB dùng reference ObjectId giữa các tài liệu thay vì lược đồ quan hệ vật lý. Hình 4.3 thể hiện quan hệ nghiệp vụ chính; các mũi tên không có nghĩa database tự áp dụng khóa ngoại.



Hình 4.3: Mô hình quan hệ giữa các collection chính

4.7.1 User và Resume

User lưu username, email, mật khẩu băm, vai trò, trạng thái tài khoản, hồ sơ ứng viên hoặc công ty và danh sách Job đã lưu. Resume tham chiếu userId, chứa dữ liệu CV có cấu trúc, mẫu và metadata. isVirtual phân biệt PDF tạm được tạo khi ứng tuyển với CV cấu trúc do người dùng quản lý; pdfData chỉ có ý nghĩa với CV PDF và bị loại khỏi truy vấn danh sách.

4.7.2 Job và Application

Job liên kết doanh nghiệp qua companyId, chứa nội dung, trạng thái và ngày hết hạn. Application liên kết Job, ứng viên và Resume. Unique index trên jobId–candidateId ngăn hồ sơ trùng. jobSnapshot lưu tiêu đề, công ty, địa điểm, loại và lương tại thời điểm nộp để lịch sử không mất ngữ cảnh khi Job bị xóa.

4.7.3 Message, Notification và trạng thái đọc

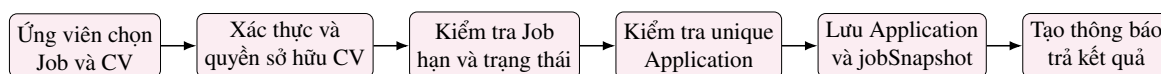
Message lưu sender, room và nội dung. ChatReadState lưu lastReadAt theo cặp user–room; HiddenChat ghi việc một phía ẩn room. Notification lưu recipient, nội dung, liên kết và isRead. Thiết kế này tách trạng thái cá nhân khỏi dữ liệu trao đổi chung.

4.7.4 AiUsage

AiUsage lưu user, feature, model, hash đầu vào, trạng thái, token, độ trễ và mã lỗi. Dữ liệu này hỗ trợ quota và quan sát kỹ thuật mà không cần lưu nguyên prompt hoặc toàn bộ CV. Hash không được dùng như cơ chế ẩn danh tuyệt đối; hệ thống vẫn cần hạn chế quyền truy cập collection thống kê.

4.8 Các quy trình nghiệp vụ chính

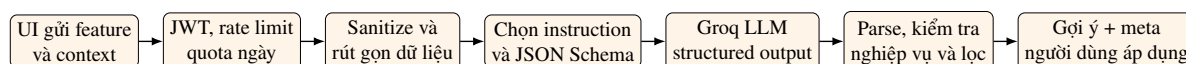
4.8.1 Ứng tuyển bằng CV



Hình 4.4: Luồng kiểm soát khi ứng tuyển bằng CV

Với PDF upload, backend kiểm tra data URL, MIME, đuôi, header và dung lượng giải mã; sau đó tạo Resume ảo và tiếp tục cùng luồng Application. Nếu bất kỳ kiểm tra nào thất bại, không tạo Resume hoặc Application mới.

4.8.2 Yêu cầu AI



Hình 4.5: Đường đi của dữ liệu trong một yêu cầu AI

Service tạo bản ghi AiUsage trước khi gọi nhà cung cấp và cập nhật trạng thái sau khi hoàn tất hoặc thất bại. Nhà cung cấp được yêu cầu áp dụng JSON Schema nghiêm ngặt; backend tiếp tục kiểm tra điều kiện ngữ nghĩa ở luồng tiểu sử và làm sạch/giới hạn đầu ra ở luồng điền CV, phỏng vấn. Frontend nhận lưu ý cùng kết quả ở các chức năng review, chấm CV và phỏng vấn. Không có bước tự động lưu kết quả AI vào Resume.

4.8.3 Chat và trạng thái đọc

Người gửi gọi REST API; server lấy định danh và tên người gửi từ tài khoản đã xác thực, kiểm tra tin có nội dung hoặc ảnh, lưu Message rồi phát newMessage đến phòng của người nhận. Văn bản chat được React hiển thị như chuỗi, còn ảnh được nén ở client trước khi chuyển thành data URL. Khi mở phòng, client cập nhật lastReadAt. Việc ẩn phòng tạo hoặc cập nhật HiddenChat theo user; Message của phía còn lại không bị xóa. Notification không dùng chung kênh này mà được tải qua API, do đó có thể có độ trễ tối đa theo chu kỳ polling.

4.9 Bảo mật và ranh giới tin cậy

Thiết kế xác định ba ranh giới tin cậy chính:

- **Giữa trình duyệt và backend:** mọi dữ liệu từ client đều không đáng tin, kể cả vai trò hoặc mã người dùng.
- **Giữa backend và dịch vụ AI:** chỉ dữ liệu đã được làm sạch, giới hạn và thật sự cần thiết mới được gửi ra ngoài.
- **Tại dữ liệu CV:** đường dẫn xem CV phải kiểm tra quan hệ sở hữu hoặc quan hệ tuyển dụng, không chỉ kiểm tra người gọi đã đăng nhập.

Các biện pháp kiểm soát hiện có được chia thành các nhóm sau:

- **Xác thực và bảo vệ HTTP:** bcryptjs, JWT trong cookie HTTP-only, Helmet, CSP, CORS theo danh sách nguồn và giới hạn tốc độ.
- **Kiểm tra dữ liệu và quyền truy cập:** kiểm tra ObjectId, danh sách trường người dùng theo ngữ cảnh, bộ làm sạch rich text/PDF/AI, chỉ mục duy nhất và truy vấn kèm điều kiện sở hữu.
- **Giới hạn thông tin vận hành:** health check công khai chỉ trả trạng thái tối giản; thông tin môi trường và cơ sở dữ liệu chi tiết yêu cầu quyền quản trị.
- **Phạm vi bảo vệ:** các biện pháp trên giúp giảm rủi ro nhưng không thay thế kiểm thử xâm nhập, phân tích CSRF, quản lý bí mật production hoặc rà soát đầy đủ mã trận quyền.

Hạn chế của thiết kế hiện tại. Backend còn tập trung nhiều route trong một file; PDF và ảnh chat dạng data URL làm tải liệu MongoDB lớn; notification dùng polling; Socket.IO trên môi trường serverless chưa phải kiến trúc phù hợp cho tải lớn. Tác vụ theo lịch chưa có kiểm thử về gọi đồng thời, retry và quan sát lỗi. Mã nguồn vẫn có khóa JWT dự phòng và cơ chế tạo tài khoản quản trị phục vụ trình diễn; chúng phải bị vô hiệu hóa hoặc bắt buộc cấu hình ở production. Đặc biệt, API gửi/ẩn chat chưa kiểm tra đầy đủ người gọi có thuộc phòng và API AI chưa chặn theo vai trò ứng viên. Phần AI còn phụ thuộc nhà cung cấp và model cấu hình, trong khi chất lượng ngữ nghĩa chưa có bộ dữ liệu chấm độc lập. Các hạn chế này được chuyển thành mục kiểm thử hoặc hướng phát triển thay vì được mô tả như chức năng đã hoàn tất.

Chương 5

Triển khai hệ thống

5.1 Tổ chức mã nguồn và phạm vi triển khai

RabbitCV được triển khai theo kiến trúc web client–server. Mã nguồn được tổ chức thành các phần chính sau:

- **Frontend:** nằm trong thư mục `src`, bao gồm `page`, `component`, `context`, `hook` và `utility`.
- **Backend:** nằm trong thư mục `server`, bao gồm Express entry point, model Mongoose, route AI, service AI, utility kiểm tra dữ liệu và tác vụ định kỳ.
- **Giao tiếp giữa hai phần:** chủ yếu sử dụng REST API; riêng thông điệp chat được phát thêm qua Socket.IO sau khi server lưu dữ liệu thành công.

Lớp	Thành phần tiêu biểu	Trách nhiệm đã triển khai
Giao diện	Page, component, React Router	Điều hướng, nhập liệu, preview CV, hiển thị Job, hồ sơ, chat và kết quả AI.
Trạng thái dùng chung	AuthContext, SavedJob- sContext, SocketContext, QueryClient	Phiên đăng nhập, cache việc đã lưu/thông báo, cập nhật lạc quan và kết nối sự kiện chat.
Nền tảng giao diện	Form schema, Radix UI, Son- ner, Error Boundary	Kiểm tra biểu mẫu, hộp thoại/menu có portal, phản hồi thao tác và phục hồi lỗi kết xuất.
API nghiệp vụ	Express routes trong <code>server.js</code>	Xác thực, Resume, Job, Application, Message, Notification, saved jobs và admin.
API AI	<code>aiRoutes.js</code> , <code>aiService.js</code>	Nắm workflow AI, quota, rate limit, timeout, structured output và ghi nhận usage.
Dữ liệu	Chín model Mongoose	User, Resume, Job, Application, Message, Notification, HiddenChat, ChatReadState và AiUsage.
Kiểm tra biên	Utility sanitizer/validator	Kiểm tra PDF, rich text CV, ngày, khả dụng của Job và dữ liệu gửi/nhận từ AI.

Bảng 5.1: Ảnh xạ các lớp triển khai của RabbitCV

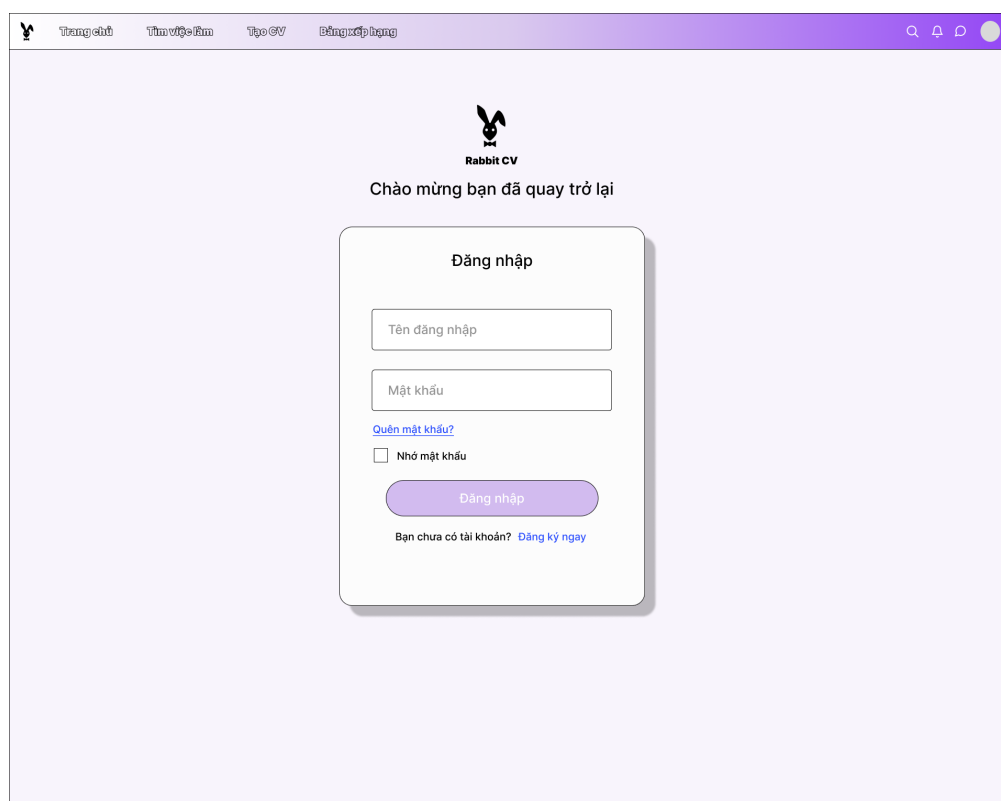
Frontend nạp lười nhiều trang để giảm lượng mã ban đầu và Vite chia các thư viện lớn thành nhóm riêng. Backend hiện vẫn tập trung phần lớn route nghiệp vụ trong `server.js`; cách tổ

chức này phù hợp bản đồ án nhưng là một giới hạn bảo trì được thảo luận ở Chương 7 và Chương 8.

5.2 Xác thực và phân quyền

Route đăng ký kiểm tra username, email, mật khẩu và vai trò trước khi tạo User; mật khẩu được băm bằng bcryptjs. Route đăng nhập tạo JWT có thời hạn một ngày hoặc 30 ngày theo lựa chọn ghi nhớ và gửi token qua cookie HTTP-only. Khi ứng dụng khởi động hoặc tải lại, AuthContext dùng bản sao trong sessionStorage cho phản hồi ban đầu, đồng thời gọi /api/me để xác nhận phiên và cung cấp currentUser cho cây component.

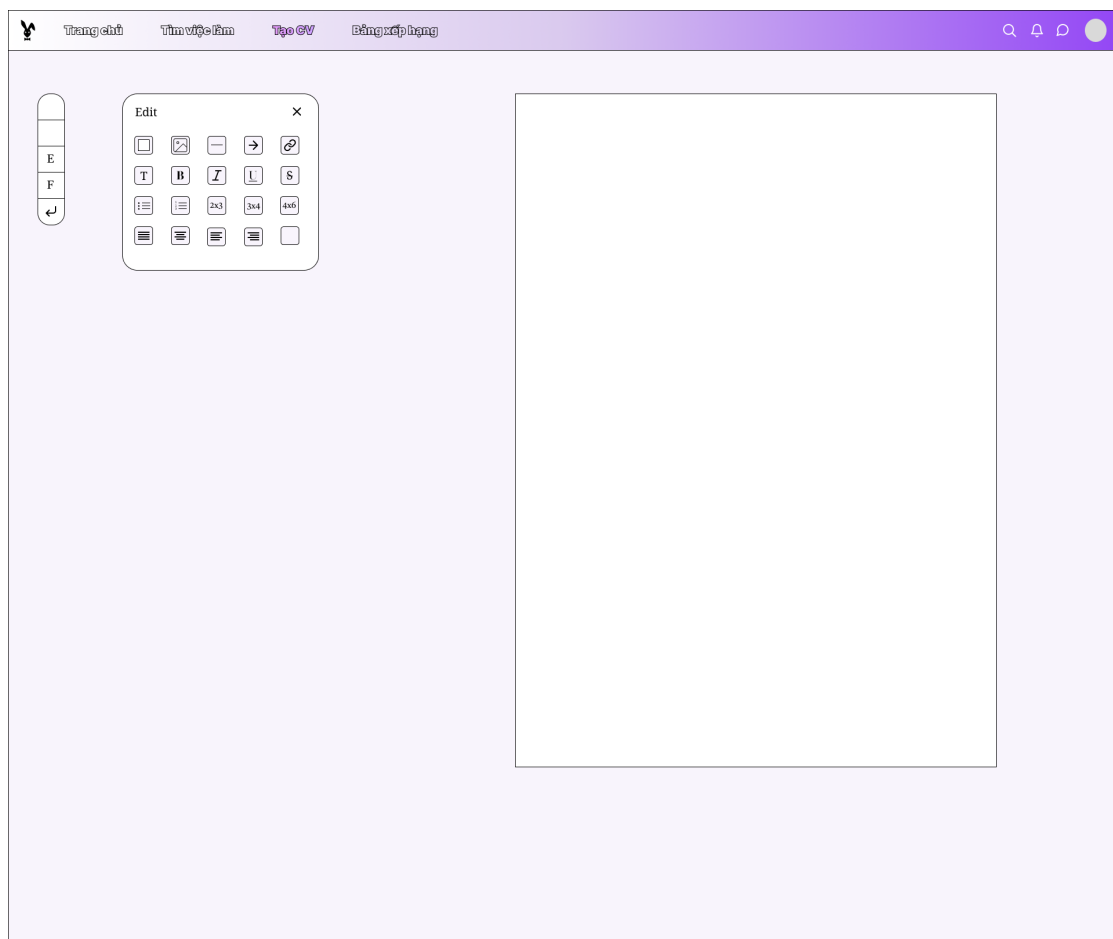
Biểu mẫu đăng nhập và đăng ký sử dụng React Hook Form cùng Zod để kiểm tra trường bắt buộc, email, xác nhận mật khẩu, vai trò và điều khoản. Ở frontend, RoleAccessGuard điều hướng doanh nghiệp và quản trị viên về cổng tương ứng; Error Boundary ở gốc cho phép tải lại khi lỗi React không được xử lý. Ở backend, middleware xác thực đọc token và các route đặc quyền kiểm tra vai trò hoặc quan hệ sở hữu. Frontend guard chỉ cải thiện trải nghiệm; quyết định cho phép đọc hoặc thay đổi dữ liệu luôn phải được thực hiện ở server.



Hình 5.1: Giao diện đăng nhập của RabbitCV

5.3 Tạo, quản lý và xuất CV

Resume lưu dữ liệu có cấu trúc theo các nhóm thông tin cá nhân, tiểu sử, kinh nghiệm, học vấn, kỹ năng, dự án, chứng chỉ, ngôn ngữ, giải thưởng và sở thích. Cách lưu này cho phép người dùng đổi template, tiếp tục chỉnh sửa và tái tạo bản trình bày mà không phải phân tích ngược một file PDF.



Hình 5.2: Khung chỉnh sửa và vùng preview CV

Các vùng mô tả dùng Tiptap. Khi lưu, backend làm sạch HTML theo danh sách thẻ cho phép; khi hiển thị, static renderer chuyển nội dung thành phần tử React thay vì chèn chuỗi HTML trực tiếp. Preview CV dùng khung có kích thước vật lý A4 và quy tắc CSS dành cho in. Hook `useResumePrint` dựa trên `react-to-print` được dùng chung ở trình biên tập và thư viện CV, đặt tiêu đề tệp, CSS trang in và xử lý lỗi. Với CV tải lên trong luồng ứng tuyển, backend kiểm tra data URL, MIME, phần mở rộng, header PDF và dung lượng giải mã tối đa 5 MB. PDF được hiển thị theo từng trang bằng `PDF.js` thay vì dùng `iframe` của trình duyệt.

Hai quyết định dữ liệu giúp luồng CV ổn định hơn. Thứ nhất, `isVirtual` phân biệt Resume dùng tạm trong một Application với CV do người dùng quản lý. Thứ hai, endpoint danh sách loại `pdfData` khỏi kết quả để tránh tải toàn bộ file base64 khi chỉ cần metadata. Endpoint xem CV kiểm tra quyền sở hữu hoặc quan hệ tuyển dụng trước khi trả nội dung.

5.4 Tích hợp AI có kiểm soát

Nhóm route `/api/ai` cung cấp năm chức năng:

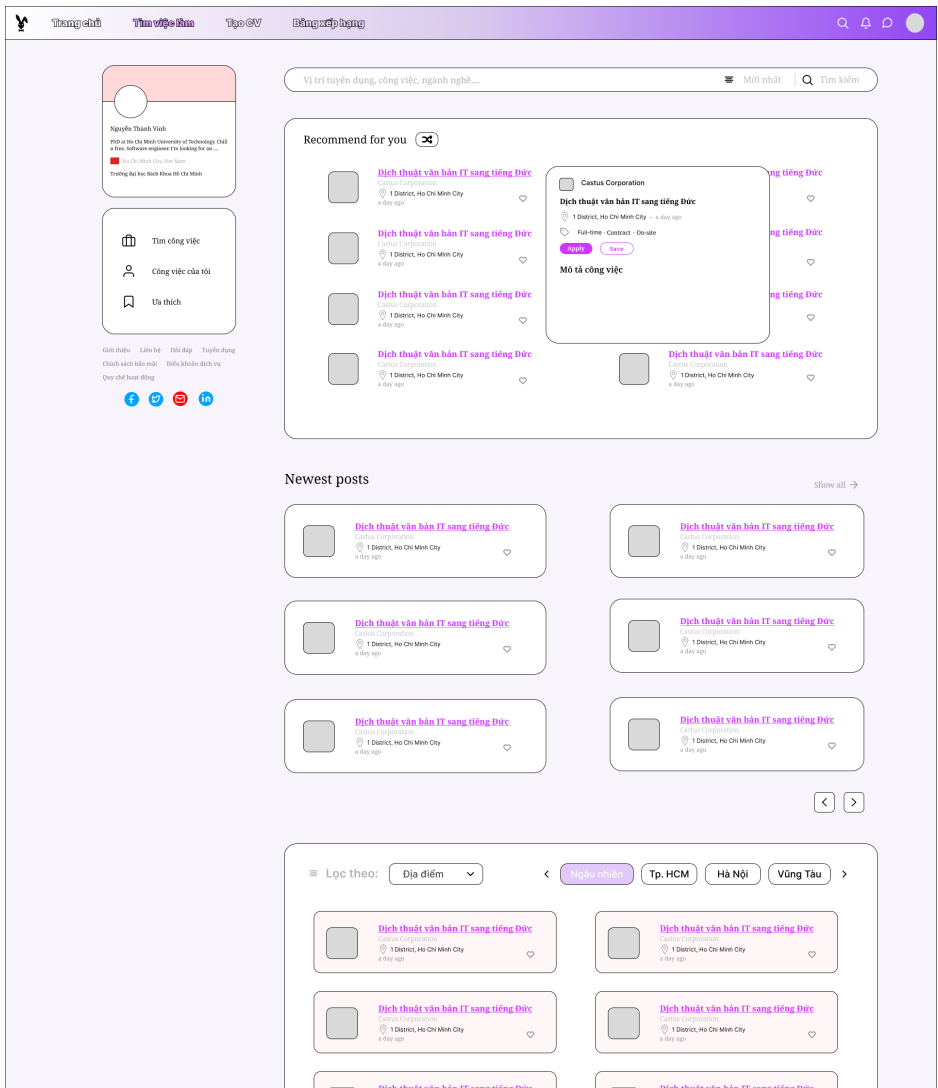
- viết hoặc cải thiện phần giới thiệu;
- rà soát CV;
- gợi ý điền các mục còn thiếu;
- so khớp CV với công việc;
- luyện tập phỏng vấn.

Tất cả yêu cầu đều đi qua backend nên khóa truy cập nhà cung cấp không xuất hiện trong frontend. Kết quả được hiển thị dưới dạng gợi ý và chỉ làm thay đổi CV khi người dùng chủ động áp dụng.

Kiến trúc, hợp đồng API, quy trình làm sạch dữ liệu, đầu ra có cấu trúc, hạn mức và thống kê sử dụng được trình bày riêng tại Chương 6.

5.5 Tìm việc và ứng tuyển

Endpoint danh sách Job là tài nguyên công khai; giao diện hỗ trợ tìm từ khóa, công ty, địa điểm và thể, đồng thời phân biệt trạng thái tin. Với người dùng đã đăng nhập, SavedJobsContext giữ giao diện dùng chung nhưng dữ liệu được quản lý bằng TanStack Query. Thao tác lưu/bỏ lưu cập nhật cache lạc quan, khôi phục trạng thái cũ khi API thất bại và làm mới lại sau khi hoàn tất.



Hình 5.3: Giao diện tìm kiếm và xem nhanh việc làm

Người dùng đã xác thực có thể nộp Resume đã lưu hoặc tải một PDF mới qua hai endpoint ứng tuyển tương ứng. Trước khi ghi dữ liệu, backend xác nhận tài khoản, quyền sở hữu CV, trạng thái và hạn của Job. Unique index trên cặp jobId-candidateId là lớp bảo vệ cuối cùng chống

ứng tuyển trùng, kể cả khi hai request đến gần nhau. API hiện chưa kiểm tra rõ vai trò phải là ứng viên; đây là điểm cần hoàn thiện thay vì chỉ dựa vào điều hướng frontend.

Application lưu jobSnapshot gồm các thông tin cần thiết của tin tại thời điểm nộp. Nhờ đó, lịch sử ứng tuyển vẫn có ngữ cảnh nếu doanh nghiệp xóa Job. Với PDF upload hợp lệ, backend tạo Resume ảo và liên kết với Application; Resume này không xuất hiện trong danh sách CV quản lý thường ngày.

5.6 Cổng doanh nghiệp

Doanh nghiệp dùng chung cơ chế xác thực nhưng có bố cục và route riêng. Các chức năng đã triển khai gồm:

- cập nhật hồ sơ công ty;
- tạo tin nháp, đăng hoặc chỉnh sửa tin;
- bật hoặc tắt nhận hồ sơ và xóa tin;
- xem danh sách ứng viên;
- cập nhật trạng thái Application.

Trạng thái hồ sơ gồm Pending, Reviewing, Interview, Offered, Rejected và Accepted. Form tin tuyển dụng bổ sung kinh nghiệm, học vấn, mô hình làm việc và kiểm tra hạn tối đa hai năm. Tuy nhiên, trạng thái xác minh doanh nghiệp chưa được kiểm tra tại endpoint tạo tin, nên quy trình duyệt chưa phải một cổng cưỡng chế hoàn chỉnh.

Khi doanh nghiệp yêu cầu xem CV, backend không chỉ kiểm tra role company mà còn kiểm tra Job/Application liên quan. Khi trạng thái Application thay đổi, hệ thống tạo Notification cho ứng viên. Cơ chế này liên kết thao tác tuyển dụng với lịch sử và thông báo, thay vì để từng màn hình thay đổi dữ liệu độc lập.

5.7 Chat và thông báo

Chat sử dụng mô hình “lưu trước, phát sau”. Client gửi nội dung qua REST API; backend lấy danh tính người gửi từ JWT, kiểm tra có văn bản hoặc ảnh, lưu Message rồi phát sự kiện newMessage đến phòng của người nhận bằng Socket.IO. Nếu ghi cơ sở dữ liệu thất bại, server không phát một thông điệp chưa tồn tại. Văn bản được React hiển thị như chuỗi; ảnh được CompressorJS nén ở client trước khi chuyển thành data URL. Backend hiện chưa áp dụng giới hạn loại/dung lượng ảnh và chưa xác minh đầy đủ người gửi thuộc phòng riêng, vì vậy không gọi đây là luồng đã được làm sạch hoàn toàn.

ChatReadState lưu lastReadAt theo người dùng và phòng để chỉ báo chưa đọc tồn tại qua lần tải lại. HiddenChat cho phép một người ẩn cuộc hội thoại khỏi danh sách của mình mà không xóa Message của phía còn lại; tin mới sau mốc ẩn có thể làm phòng xuất hiện lại. Notification là collection riêng, hỗ trợ phân trang, đếm chưa đọc, đánh dấu từng mục hoặc tắt cả và xóa. Hook thông báo dùng TanStack Query để cache các trang, cập nhật trạng thái đọc trước trên giao diện và truy vấn số chưa đọc mỗi 30 giây; message dùng kết nối thời gian thực.

5.8 Khu vực quản trị

Backend cung cấp các nhóm chức năng quản trị sau:

- thống kê và chẩn đoán health chi tiết;

- xem danh sách, ẩn, hiện hoặc xóa Job;
- xem danh sách và xác minh doanh nghiệp;
- khóa hoặc xóa User.

Các endpoint này dùng xác thực và kiểm tra vai trò admin. Frontend hiện chỉ giữ các màn hình có API thật gồm dashboard, người dùng, việc làm và doanh nghiệp; các đường dẫn báo cáo, đánh giá và nhật ký cũ chuyển về trang hợp lệ. Endpoint khóa User hiện chỉ đặt trạng thái khóa, chưa có thao tác mở lại tương ứng cho tài khoản thông thường. Vì vậy báo cáo ghi nhận quản trị ở mức cơ bản, không xem toàn bộ khu vực admin là hoàn tất.

5.9 Bảo mật và giới hạn triển khai

Các lớp bảo vệ chính trong quá trình triển khai gồm:

- **HTTP header:** Helmet thiết lập CSP, chặn nhúng trang và đặt chính sách referrer; cấu hình Vercel bổ sung các header về quyền truy cập thiết bị, kiểu nội dung và tài nguyên chéo nguồn.
- **Nguồn truy cập và tần suất:** CORS chỉ cho các origin được cấu hình; API chung chịu giới hạn 300 yêu cầu trong 15 phút, đăng nhập tối đa 10 lần thất bại trong 15 phút, đăng ký tối đa 10 lần mỗi giờ và AI có giới hạn riêng.
- **Dữ liệu phản hồi:** các danh sách trường SELF_USER_FIELDS, PUBLIC_USER_FIELDS và ADMIN_USER_FIELDS giới hạn dữ liệu tài khoản được trả theo từng ngữ cảnh; mật khẩu và mã PIN không được đưa vào phản hồi.

Các biện pháp trên là lớp bảo vệ cơ bản, chưa phải chứng nhận an toàn. Khi thiếu biến môi trường, mã nguồn vẫn có khóa JWT dự phòng; kết nối cơ sở dữ liệu còn tự tạo tài khoản quản trị trình diễn với mật khẩu ngẫu nhiên. Hai hành vi này thuận tiện cho phát triển nhưng không phù hợp production. Ngoài ra cần hoàn thiện kiểm tra vai trò cho AI/ứng tuyển/lưu việc, xác minh thành viên phòng chat ở mọi thao tác, CSRF, chính sách mật khẩu, giới hạn ảnh, nhật ký kiểm toán và quy trình xoay bí mật.

5.10 Cấu hình và khả năng vận hành

Trong môi trường phát triển, Vite phục vụ frontend và proxy request /api cùng /socket.io đến backend Express. Script start:all điều phối hai tiến trình; Nodemon chỉ theo dõi backend và tệp môi trường để build frontend không gây khởi động lại máy chủ. Backend đọc URI MongoDB, tên cơ sở dữ liệu, JWT secret, nguồn frontend, cấu hình AI và khóa cron từ biến môi trường. Tên biến và quy trình chạy được tổng hợp ở Phụ lục A; báo cáo không ghi giá trị bí mật.

Tệp vercel.json định tuyến API và Socket.IO đến Express, phục vụ ứng dụng một trang, đặt security header và khai báo lịch 0 0 * * * cho endpoint hạn tin. Lệnh build Vite tạo được sản phẩm frontend production với các nhóm mã tách riêng. Kết quả này xác nhận mã có thể đóng gói, nhưng không đồng nghĩa hệ thống đã được đánh giá tải, HTTPS, cơ sở dữ liệu production, độ bền Socket.IO serverless hoặc quy trình khôi phục sự cố. Triển khai thực tế cần bí mật mạnh, CORS đúng nguồn, cookie Secure, quan sát log và chiến lược lưu tệp ngoài MongoDB.

5.11 Đối chiếu yêu cầu với hiện thực

Nhóm yêu cầu	Bảng chứng triển khai	Điểm cần xác minh ở Chương 7
FR-CV, FR-AI	Resume, A4/PDF, năm route AI, AiUsage	Layout A4, PDF validation, sanitizer và hợp đồng AI.
FR-JOB, FR-APP	Job, Application, saved jobs, unique index, snapshot	Hạn tin, trùng hồ sơ, lịch sử và dữ liệu PDF.
FR-COM	Route Job/Application và layout doanh nghiệp	Trạng thái hồ sơ, quyền xem CV và notification.
FR-MSG	Message, Socket.IO, ChatReadState, HiddenChat	Rich text, badge chưa đọc và trạng thái theo user.
FR-ADM	Admin routes và dashboard	Ma trận quyền cùng sự tương ứng frontend-backend.

Bảng 5.2: Truy vết từ yêu cầu sang thành phần triển khai

Chương 6

Mô-đun trí tuệ nhân tạo

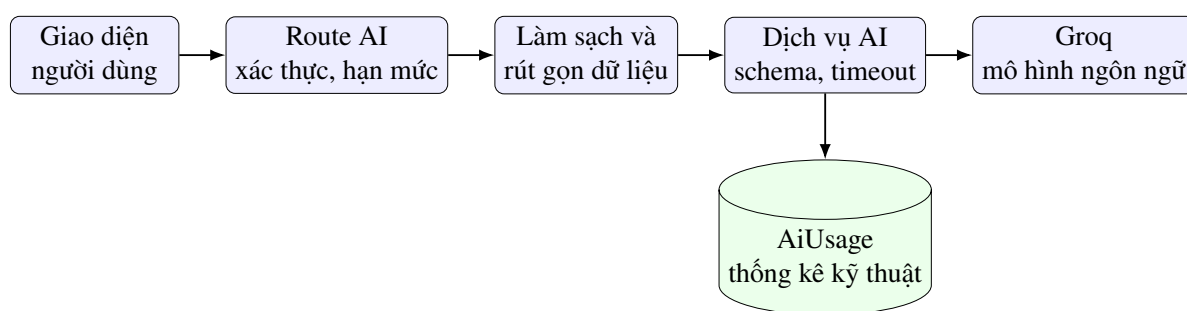
Mô-đun trí tuệ nhân tạo của RabbitCV hỗ trợ người dùng xây dựng và đánh giá nội dung CV trong ngữ cảnh tìm việc. Mô-đun không tự động thay đổi dữ liệu đã lưu và không thay thế quyết định của người dùng hoặc nhà tuyển dụng. Chương này trình bày phạm vi, cấu trúc, năng lực, quy trình kiểm soát và các giới hạn của mô-đun.

6.1 Mục tiêu và phạm vi

Mục tiêu của mô-đun là giảm thời gian soạn nội dung, giúp người dùng nhận ra điểm thiếu trong CV và tạo môi trường luyện tập trước phỏng vấn. Dữ liệu đầu vào có thể gồm nội dung CV, mô tả công việc, chế độ xử lý và lịch sử hội thoại. Kết quả được trả về dưới dạng dữ liệu có cấu trúc để giao diện hiển thị thành từng nhóm gợi ý.

AI chỉ đóng vai trò hỗ trợ. Nội dung sinh ra có thể chung chung, suy diễn quá mức hoặc chưa phù hợp với kinh nghiệm thực tế. Vì vậy, RabbitCV yêu cầu người dùng chủ động áp dụng gợi ý; điểm phù hợp CV–công việc không được dùng để tự động loại ứng viên.

6.2 Kiến trúc tổng thể của mô-đun



Hình 6.1: Kiến trúc xử lý của mô-đun trí tuệ nhân tạo

6.2.1 Thành phần giao diện và lớp API

Người dùng kích hoạt chức năng AI từ trình biên tập CV, cửa sổ rà soát, trang việc làm hoặc trang luyện phỏng vấn. Frontend chỉ gửi dữ liệu cần thiết đến nhóm đường dẫn `/api/ai`; khóa truy cập nhà cung cấp không được đưa vào mã giao diện.

Các route AI kiểm tra phiên đăng nhập, giới hạn tốc độ, hạn mức theo ngày và dữ liệu của từng chức năng. Nếu yêu cầu không hợp lệ, backend trả mã lỗi trước khi gọi mô hình, nhờ đó tránh sử dụng tài nguyên AI cho dữ liệu không thể xử lý.

6.2.2 Bộ làm sạch và dịch vụ AI

Bộ làm sạch chuyển CV, công việc và lịch sử phỏng vấn thành ngữ cảnh rút gọn. Dịch vụ AI chọn cấu hình, chỉ dẫn và JSON Schema theo chức năng, sau đó dùng OpenAI SDK để gửi yêu cầu đến API tương thích OpenAI của Groq. Thời gian chờ và số lần thử lại được giới hạn để một yêu cầu lỗi không giữ tài nguyên máy chủ quá lâu.

Sau khi nhận kết quả, dịch vụ phân tích chuỗi JSON, thực hiện các kiểm tra hoặc bước làm sạch bổ sung rồi trả dữ liệu cho route. Dữ liệu có cấu trúc giúp giao diện hiển thị từng điểm số, vấn đề hoặc gợi ý mà không phải tự tách một đoạn văn bản tự do.

6.2.3 Lưu thông kê sử dụng

Mỗi lần gọi tạo một bản ghi AiUsage trước khi liên hệ nhà cung cấp. Sau khi hoàn tất hoặc thất bại, bản ghi được cập nhật với số token, độ trễ, trạng thái và mã lỗi. Dữ liệu này hỗ trợ tính hạn mức và quan sát vận hành mà không cần lưu nguyên văn toàn bộ CV hoặc prompt.

6.3 Các chức năng AI

Endpoint	Chức năng	Kết quả có cấu trúc
GET /api/ai/status	Kiểm tra khả dụng	Trạng thái bật hoặc tắt, mô hình, hạn mức ngày và số lượt còn lại.
POST /api/ai/resume/improve	Viết hoặc cải thiện giới thiệu	Nội dung đề xuất theo chế độ viết mới, cải thiện hoặc rút gọn.
POST /api/ai/resume/review	Rà soát CV	Điểm tổng quan, điểm thành phần, điểm mạnh và vấn đề theo mức ưu tiên.
POST /api/ai/resume/fill	Gợi ý điền CV	Danh sách trường và nội dung đề xuất có giới hạn.
POST /api/ai/resume/score	So khớp CV-công việc	Điểm kỹ năng, kinh nghiệm, học vấn, từ khóa và khuyến nghị.
POST /api/ai/interview/turn	Luyện phỏng vấn	Câu hỏi kế tiếp hoặc phần tổng kết buổi luyện tập.

Bảng 6.1: Các endpoint của mô-đun trí tuệ nhân tạo

6.3.1 Viết và cải thiện phần giới thiệu

Chức năng nhận phần giới thiệu hiện tại, chức danh mục tiêu, kỹ năng và một phần ngữ cảnh nghề nghiệp. Người dùng có thể yêu cầu viết mới, cải thiện hoặc rút gọn. Kết quả là một đoạn giới thiệu đề xuất; nội dung chỉ được đưa vào CV sau khi người dùng xác nhận.

6.3.2 Rà soát nội dung CV

Chức năng rà soát phân tích CV theo nhiều tiêu chí và trả điểm tổng quan, điểm thành phần, điểm mạnh cùng danh sách vấn đề. Mỗi vấn đề có mức độ ưu tiên và gợi ý xử lý để người dùng

biết phần nào cần xem trước. Điểm số dùng để định hướng chỉnh sửa, không phải kết luận tuyển dụng.

6.3.3 Gợi ý bổ sung các mục còn thiếu

Chức năng điền CV xác định những phần còn thiếu hoặc có ít thông tin, sau đó tạo danh sách đề xuất theo các trường được cho phép. Backend giới hạn số lượng và độ dài của từng đề xuất. Giao diện không tự động ghi đề dữ liệu đã lưu; người dùng chọn nội dung phù hợp trước khi áp dụng.

6.3.4 Đánh giá mức độ phù hợp giữa CV và công việc

Chức năng so khớp nhận ngữ cảnh CV và mô tả việc làm, sau đó trả các nhóm điểm về kỹ năng, kinh nghiệm, học vấn và từ khóa. Kết quả kèm nhận xét về khoảng thiếu và hành động có thể thực hiện. Chức năng này hỗ trợ ứng viên điều chỉnh hồ sơ, không được sử dụng như quyết định tự động của doanh nghiệp.

6.3.5 Luyện tập phỏng vấn

Mỗi lượt luyện tập gửi vai trò ứng tuyển, ngữ cảnh CV, công việc và lịch sử trả lời đã được giới hạn. Mô hình trả câu hỏi tiếp theo hoặc phần tổng kết gồm điểm mạnh, điểm cần luyện và khuyến nghị. Lịch sử chỉ giữ lượng hội thoại cần thiết cho tối đa 12 câu để kiểm soát kích thước yêu cầu.

6.4 Quy trình xử lý một yêu cầu AI

6.4.1 Xác thực, giới hạn tốc độ và hạn mức

Mọi chức năng AI yêu cầu phiên đăng nhập hợp lệ. Nhóm route AI giới hạn 30 yêu cầu trong 15 phút và kiểm tra số lượt đã dùng trong ngày. Hạn mức mặc định là 100 yêu cầu mỗi ngày cho một người dùng, nhưng có thể thay đổi bằng biến môi trường. Endpoint trạng thái cho phép backend thông báo mô-đun có khả dụng hay không và số lượt còn lại.

6.4.2 Làm sạch và rút gọn dữ liệu

CV gửi đến mô hình không giữ nguyên toàn bộ tài liệu. Thông tin liên hệ trực tiếp được thay bằng trạng thái đã điền hoặc chưa điền; các danh sách được giới hạn số phần tử và văn bản được giới hạn độ dài. Dữ liệu công việc và lịch sử phỏng vấn cũng được lọc theo các trường cần cho chức năng tương ứng.

Biện pháp này giảm dữ liệu truyền ra nhà cung cấp, hạn chế yêu cầu quá lớn và giảm khả năng nội dung không liên quan chi phối kết quả. Tuy nhiên, làm sạch dữ liệu không thể loại bỏ hoàn toàn prompt độc hại hoặc suy diễn sai của mô hình.

6.4.3 Tạo yêu cầu và đầu ra có cấu trúc

Dịch vụ chọn chỉ dẫn hệ thống và JSON Schema dựa trên mã chức năng. OpenAI SDK được cấu hình với địa chỉ Groq, khóa API, mô hình và thời gian chờ. Giá trị mặc định hiện tại là mô

hình `openai/gpt-oss-120b` và thời gian chờ 20 giây; cấu hình có thể thay đổi theo môi trường triển khai.

Nhà cung cấp được yêu cầu trả đầu ra theo JSON Schema ở chế độ nghiêm ngặt. Backend phân tích JSON và áp dụng thêm điều kiện nghiệp vụ hoặc hàm làm sạch theo từng chức năng. Vì chưa có một bộ kiểm tra JSON Schema cục bộ độc lập cho mọi phản hồi, báo cáo không xem đầu ra có cấu trúc là bảo đảm cho tính đúng đắn về ngữ nghĩa.

6.4.4 Trả kết quả và quyền quyết định của người dùng

Kết quả được gửi về giao diện cùng thông tin cần thiết để trình bày điểm số, gợi ý hoặc trạng thái lỗi. Những chức năng thay đổi nội dung CV đều yêu cầu thao tác áp dụng riêng. Nếu yêu cầu thất bại hoặc hết thời gian chờ, dữ liệu CV đang lưu không bị thay đổi.

6.5 Quản lý thống kê sử dụng

AiUsage lưu mã người dùng, chức năng, mô hình, mã băm đầu vào, token đầu vào, token đầu ra, độ trễ, trạng thái và lỗi kỹ thuật. Mã băm hỗ trợ nhận diện yêu cầu ở mức thống kê nhưng không cho phép khôi phục trực tiếp nội dung CV. Chỉ mục TTL xóa bản ghi sau 90 ngày để giới hạn thời gian lưu dữ liệu quan sát.

Thống kê sử dụng phục vụ ba mục đích chính:

- tính hạn mức theo ngày;
- theo dõi độ trễ hoặc lỗi của nhà cung cấp;
- tạo bằng chứng vận hành cho từng chức năng.

Hệ thống không cần lưu nguyên prompt hoặc toàn bộ CV trong bản ghi AiUsage.

6.6 Kiểm soát rủi ro và bảo vệ dữ liệu

Mô-đun áp dụng nhiều lớp kiểm soát:

- xác thực người dùng;
- giới hạn tốc độ và hạn mức sử dụng;
- giới hạn thời gian chờ;
- làm sạch dữ liệu đầu vào;
- yêu cầu đầu ra có cấu trúc;
- kiểm tra kết quả sau phản hồi.

Khóa nhà cung cấp nằm ở backend và được đọc từ biến môi trường. Các biện pháp này phù hợp với hướng quản lý rủi ro AI minh bạch, có thể kiểm tra và lấy con người làm trung tâm [12].

Dù vậy, kết quả của mô hình không mặc nhiên chính xác. Người dùng cần đối chiếu gợi ý với kinh nghiệm thực tế, tránh thêm kỹ năng hoặc thành tích không có thật và kiểm tra nội dung trước khi xuất CV. Hệ thống không dùng điểm AI để tự động quyết định việc ứng tuyển hoặc tuyển dụng.

6.7 Hạn chế của mô-đun

Chất lượng kết quả phụ thuộc vào độ đầy đủ của CV và mô tả công việc. Mô hình có thể trả gợi ý chung chung, chấm điểm thiếu ổn định hoặc suy diễn thông tin không có trong dữ liệu. Hệ

thông cũng phụ thuộc vào kết nối mạng, hạn mức và khả năng sẵn sàng của nhà cung cấp bên ngoài.

Các kiểm thử hiện tại xác minh hợp đồng API, quy tắc làm sạch, hạn mức và cấu trúc xử lý nhưng chưa phải đánh giá đầy đủ chất lượng ngữ nghĩa trên một bộ CV và công việc có nhãn chuyên gia. Vì vậy, điểm số và nhận xét AI được xem là thông tin tham khảo cần tiếp tục đánh giá trong môi trường sử dụng thực tế.

6.8 Tổng kết chương

Mô-đun AI của RabbitCV gồm năm chức năng được triển khai qua một chuỗi xử lý chung từ xác thực, làm sạch dữ liệu, gọi mô hình, kiểm soát đầu ra đến lưu thống kê. Việc tách lớp route, bộ làm sạch, dịch vụ và hợp đồng đầu ra giúp các chức năng dùng chung quy tắc bảo vệ. Vai trò quyết định cuối cùng vẫn thuộc về người dùng, còn kết quả AI được giới hạn ở mức hỗ trợ soạn thảo, đánh giá và luyện tập.

Chương 7

Kiểm thử và đánh giá

7.1 Mục tiêu và nguyên tắc đánh giá

Việc đánh giá tập trung vào các rủi ro chính của đề tài:

- CV phải giữ cấu trúc A4;
- PDF tải lên phải được kiểm tra;
- Job không khả dụng không được nhận hồ sơ;
- dữ liệu lịch sử không mất khi Job bị xóa;
- nội dung rich text và dữ liệu AI không được mang mã thực thi;
- kết quả AI phải tuân theo hợp đồng cấu trúc;
- trạng thái đọc phải tồn tại qua lần tải lại.

Báo cáo phân biệt ba mức bằng chứng:

- **Đã chạy:** kết quả được lấy từ lệnh thực thi trong môi trường đánh giá.
- **Đối chiếu mã nguồn:** xác nhận thành phần hoặc điều kiện tồn tại nhưng chưa chứng minh luồng chạy toàn hệ thống.
- **Kịch bản đề xuất:** kiểm thử thủ công cần thực hiện trước nghiệm thu; báo cáo không tự gán kết quả đạt cho các kịch bản chưa chạy.

7.2 Môi trường và khả năng tái lập

Thông tin	Giá trị ghi nhận
Thời điểm đánh giá	Ngày 09/08/2026, múi giờ Việt Nam.
Mã nguồn sản phẩm	Nhánh main, working tree tại D:\Github\DA2; commit gần nhất dcc8c58.
Trạng thái nguồn	Working tree có cả thay đổi đã theo dõi và tệp chưa theo dõi tại thời điểm ghi nhận; riêng hash commit không đủ tái tạo phiên bản đã kiểm tra.
Runtime	Node.js 26.5.0; npm 11.17.0.
Chuỗi kiểm tra	npm run check, lần lượt chạy ESLint, Node Test Runner và Vite production build.
Phạm vi không dùng	Không gọi API AI trả phí để chấm ngữ nghĩa; không chạy tải, pentest hoặc browser end-to-end trong lần đánh giá này.

Bảng 7.1: Môi trường đánh giá mã nguồn DA2

Việc ghi rõ working tree chưa sạch là cần thiết: thay đổi hiện tại trải rộng từ dependency, cấu hình Vite/Vercel, giao diện, route, service, model đến test và tài liệu, nhưng chưa thuộc commit dcc8c58. Để tái lập tuyệt đối, phiên bản sản phẩm cần được commit hoặc phát hành bằng tag sau khi loại tệp tạm và bí mật.

7.3 Kết quả quy trình kiểm tra tự động

Bước	Lệnh con	Kết quả ghi nhận	Đánh giá
Phân tích tĩnh	npm run lint	ESLint hoàn tất, 0 lỗi và 0 cảnh báo.	Đạt
Kiểm thử logic	npm test	41/41 trường hợp đạt, 0 thất bại.	Đạt
Đóng gói frontend	npm run build	Vite 7.3.6 build thành công, 1.906 module được biến đổi; không phát cảnh báo kích thước chunk.	Đạt
Toàn chuỗi	npm run check	Cả ba bước hoàn tất với mã thoát thành công.	Đạt

Bảng 7.2: Kết quả chạy quy trình kiểm tra ngày 09/08/2026

So với mốc trước, hai cảnh báo React Hook đã được loại bỏ và trang báo cáo quản trị không còn dùng dữ liệu minh họa. Các trang được nạp lười; Vite tách riêng nhóm PDF, rich text, React, giao diện, form, query, router, thời gian thực và ngày giờ. Ba nhóm JavaScript lớn nhất trong output lần này là PDF khoảng 433 kB, rich text khoảng 428 kB và React khoảng 385 kB; PDF worker là tài nguyên riêng khoảng 1,26 MB. Build sạch xác nhận khả năng đóng gói, không tự chứng minh thời gian tải hoặc hiệu năng mạng đạt yêu cầu.

7.4 Kiểm kê bộ kiểm thử

Mười hai tệp test sử dụng Node Test Runner, tổng cộng 41 trường hợp. Nội dung dưới đây được tổng hợp theo đúng tên test trong mã nguồn.

Tệp test	Số	Phạm vi chính
a4Layout.test.js	3	Kích thước A4, hành động tải PDF dựa trên print và render PDF upload không dùng native iframe.
aiDataSanitizer.test.js	5	Loại HTML thực thi, giới hạn nội dung, tách lời khuyên khỏi CV, bỏ thông tin liên hệ và kỹ năng chứa mã nguy hiểm.
aiIntegration.test.js	4	Route AI có xác thực, structured output, các workflow frontend, trang phỏng vấn và điều hướng theo role.
companyUi.test.js	3	Nhân tiếng Việt, chuẩn hóa thông tin Job và định dạng ngày trên hồ sơ công khai.
dateInput.test.js	5	Chuẩn hóa ngày cũ, ngày Resume, hoạt động đang diễn ra và chuyển đổi biểu diễn tháng.
interviewDataSanitizer.test.js	3	Giới hạn thiết lập, làm sạch lịch sử 12 câu và giới hạn feedback đầu ra.

Bảng 7.3: Danh mục tệp kiểm thử tự động (phần 1)

Tệp test	Số	Phạm vi chính
jobAvailability.test.js	3	Job đã xóa, Job active còn hạn và Job hết hạn.
jobDeadlineUi.test.js	3	Ưu tiên trạng thái hết hạn, chặn ứng tuyển trước khi đồng bộ và cho phép Job còn hạn.
pdfValidation.test.js	3	Nhận PDF data URL, từ chối Word/nội dung giả và chặn file lớn hơn 5 MB.
readStatusPersistence.test.js	2	Lưu trạng thái đọc message và notification ở server thay vì chỉ ẩn badge trong React state.
resumeAiFill.test.js	4	Merge kỹ năng an toàn, chỉ điền mục trống và giữ mô tả kỹ thuật hợp lệ.
richTextSanitizer.test.js	3	Giữ định dạng CV được hỗ trợ, loại HTML nhúng/thực thi và xử lý input không phải chuỗi.
Tổng	41	Toàn bộ 41 trường hợp đạt.

Bảng 7.4: Danh mục tệp kiểm thử tự động (phần 2)

7.5 Truy vết yêu cầu và bằng chứng

Yêu cầu	Bằng chứng tự động	Mức bao phủ
FR-CV-01, FR-CV-02	a4Layout, dateInput, richTextSanitizer	Bao phủ layout, ngày và làm sạch nội dung CV; chưa bao phủ CRUD qua database thật.
FR-CV-03	pdfValidation, a4Layout	Bao phủ validator và cơ chế render; chưa đo PDF đa trang trên nhiều trình duyệt.
FR-AI-01-03	aiDataSanitizer, aiIntegration, resumeAiFill, interviewDataSanitizer	Bao phủ hợp đồng, sanitizer và wiring; chưa chấm chất lượng ngữ nghĩa từ provider.
FR-JOB-01, FR-APP-01	jobAvailability, jobDeadlineUi	Bao phủ khả dụng Job; unique index và transaction cần integration test với MongoDB.
FR-APP-02	Đối chiếu model Application và logic jobSnapshot.	Có bằng chứng mã nguồn, chưa có test tự động chuyên biệt.
FR-COM-01	companyUi	Bao phủ một phần UI; chưa bao phủ toàn bộ vòng đời Job/Application.
FR-MSG-01	readStatusPersistence	Bao phủ trạng thái đọc ở mức mã; chưa kiểm thử quyền phòng, ảnh chat hoặc Socket.IO end-to-end.
FR-ADM-01	Điều hướng vai trò trong aiIntegration; ESLint chạy sạch.	Bao phủ hạn chế, cần kiểm thử ma trận quyền và API bằng tài khoản thật.

Bảng 7.5: Ma trận truy vết yêu cầu–kiểm thử

7.6 Kịch bản kiểm thử thủ công cần thực hiện

Các kịch bản dưới đây là checklist nghiệm thu, chưa được ghi nhận là đã chạy trong phiên đánh giá ngày 09/08/2026.

Nhóm	Kịch bản và tiêu chí chấp nhận	Trạng thái
Xác thực/RBAC	Đăng nhập bằng ứng viên, doanh nghiệp, admin; thử URL và API sai vai trò; tải lại phiên; đăng xuất. Kiểm tra riêng AI, lưu việc, ứng tuyển và phòng chat ở backend.	Chưa chạy thử công
CV/A4/PDF	Tạo CV ngắn và dài, in bằng các trình duyệt mục tiêu, đo trang 210 x 297 mm; mở PDF nhiều trang; thử file giả, sai MIME và quá 5 MB.	Chưa chạy thử công
Ứng tuyển	Nộp bằng Resume và PDF, gửi lặp, thử Job draft/expired/deleted; xóa Job rồi kiểm tra lịch sử từ snapshot.	Chưa chạy thử công
Doanh nghiệp	Tạo/sửa/đóng/mở/xóa Job, xem CV đúng và sai quan hệ, chuyển toàn bộ trạng thái Application.	Chưa chạy thử công
Chat/thông báo	Gửi giữa hai trình duyệt, thử phòng không thuộc người gọi, ảnh lớn/sai loại, mất và kết nối lại mạng, đánh dấu đọc, ẩn phòng một phía.	Chưa chạy thử công
AI	Thử nắm workflow, lỗi thiếu khóa, timeout, hết quota và payload nguy hiểm; xác nhận AI không tự ghi đề Resume.	Chưa chạy thử công

Bảng 7.6: Checklist nghiệm thu thủ công chưa thực hiện

7.7 Đánh giá riêng cho chức năng AI

Kết quả 16 test thuộc bốn tệp tập trung vào AI xác nhận được các thuộc tính có tính quyết định: dữ liệu được làm sạch và giới hạn, route yêu cầu xác thực, nhà cung cấp được yêu cầu trả structured output, kết quả fill không ghi thêm sự kiện vào mục đã có và feedback phỏng vấn bị giới hạn. Ba test rich text bổ sung bằng chứng cho nội dung CV nhưng được tính ở nhóm riêng. Đây là bằng chứng về đường nối và hợp đồng phần mềm, không phải bằng chứng rằng nội dung do mô hình sinh luôn đúng hoặc hữu ích.

Để đánh giá chất lượng ngữ nghĩa, quy trình cần thực hiện các bước sau:

- Xây dựng bộ dữ liệu cố định gồm CV ở nhiều mức kinh nghiệm và mô tả công việc tương ứng.
- Ẩn danh thông tin cá nhân trong dữ liệu đánh giá.
- Chạy cùng mô hình, prompt và tham số cho toàn bộ mẫu.
- Có ít nhất hai người chấm độc lập theo các tiêu chí:
 - bám sát dữ liệu nguồn;
 - tính đúng đắn;
 - khả năng hành động;
 - độ rõ ràng;
 - không bịa đặt;
 - không tạo phân biệt đối xử.

Phụ lục A cung cấp rubric đề xuất. Chỉ sau bước này mới nên báo cáo điểm trung bình, độ đồng thuận giữa những người chấm hoặc kết quả so sánh mô hình.

7.8 Hạn chế và rủi ro còn lại

- Bộ test thiên về unit và source-level integration; chưa chạy MongoDB thật, browser end-to-end, ma trận trình duyệt A4 hoặc Socket.IO khi mạng chập chờn.
- Working tree DA2 chưa có commit/tag tái lập; chưa có CI, số liệu bao phủ mã hoặc artifact kiểm thử được lưu theo phiên bản.
- Kiểm tra quyền theo vai trò chưa đồng đều: API AI, ứng tuyển và lưu việc chủ yếu dựa trên JWT; gửi/ảnh chat chưa xác minh đầy đủ người gọi thuộc phòng.
- Khóa JWT dự phòng và tài khoản quản trị seed chỉ phù hợp phát triển; ảnh chat chưa có validator server tương đương PDF.
- Chất lượng ngữ nghĩa, tính công bằng và chi phí AI chưa được chăm trên bộ dữ liệu độc lập; chưa gọi provider thật trong chuỗi kiểm tra tự động.
- Chưa thực hiện kiểm thử tải, kiểm thử xâm nhập, CSRF chuyên sâu, accessibility audit, Vercel Cron/Socket.IO production hoặc khôi phục cơ sở dữ liệu và tệp.

Kết luận đánh giá. Trong phạm vi đã chạy, RabbitCV đạt 41/41 test, ESLint không có lỗi hoặc cảnh báo và build frontend thành công mà không phát cảnh báo kích thước chunk. Đây là baseline kỹ thuật của working tree ngày 09/08/2026, chưa phải bằng chứng sẵn sàng production; kiểm thử thủ công, tích hợp MongoDB, ma trận quyền, hạ tầng triển khai và đánh giá ngữ nghĩa AI vẫn cần hoàn tất.

Chương 8

Kết luận và hướng phát triển

8.1 Mức độ hoàn thành mục tiêu

Đồ án đã xây dựng RabbitCV với trọng tâm là tạo CV và hỗ trợ nội dung bằng AI; các chức năng tuyển dụng đóng vai trò cung cấp ngữ cảnh sử dụng CV. Mức độ hoàn thành được tổng hợp từ thành phần triển khai và kết quả đánh giá ở Chương 7, không chỉ từ việc giao diện có thể hiển thị.

Mục tiêu	Kết quả và bằng chứng	Kết luận
Tạo và quản lý CV	Resume có cấu trúc, template, preview A4, in/tải PDF; test A4 và ngày tháng đạt.	Hoàn thành trong phạm vi
AI hỗ trợ CV	Viết/cải thiện tiểu sử, review, fill, so khớp CV–Job và phỏng vấn; có sanitizer, schema, quota và usage log.	Hoàn thành mức demo
Tìm việc và ứng tuyển	Tìm/lưu Job, nộp Resume hoặc PDF, unique index, trạng thái và jobSnapshot.	Hoàn thành trong phạm vi
Kết nối hai phía	Cổng doanh nghiệp, xử lý Application, chat Socket.IO, trạng thái đọc/ẩn và notification polling.	Hoàn thành chính
Quản trị	Có thống kê và nhóm endpoint quản lý User, Company, Job.	Hoàn thành một phần
Nền tảng giao diện	Query cache, biểu mẫu theo schema, hộp thoại/menu dùng chung, toast, Error Boundary và hook in CV.	Hoàn thành đợt 1
Kiểm chứng kỹ thuật	41/41 test đạt, ESLint 0 lỗi và 0 cảnh báo, build 1.906 module thành công, không có cảnh báo kích thước chunk.	Đạt baseline

Bảng 8.1: Tổng hợp mức độ hoàn thành mục tiêu

Kết quả nổi bật của đồ án không nằm ở số lượng màn hình mà ở việc nối được vòng đời CV với nghiệp vụ sử dụng: dữ liệu CV được chỉnh sửa có cấu trúc, được trình bày theo A4, có thể nhận gợi ý AI dưới quyền quyết định của người dùng, rồi được dùng để ứng tuyển và trao đổi với doanh nghiệp.

8.2 Đóng góp kỹ thuật

Các đóng góp kỹ thuật chính của đồ án gồm:

- **Tích hợp AI có kiểm soát:** Frontend không giữ khóa nhà cung cấp; backend xác thực, giới hạn tần suất và hạn mức, làm sạch dữ liệu, yêu cầu JSON Schema và kiểm tra hoặc lọc kết quả theo từng luồng. Cơ chế “gợi ý rồi mới áp dụng” tránh để mô hình tự sửa hồ sơ cá nhân.
- **Bảo toàn và giới hạn dữ liệu:** Bản chụp việc làm giữ ngữ cảnh lịch sử; chỉ mục duy nhất ngăn ứng tuyển trùng; CV tạm tách PDF ứng tuyển khỏi thư viện CV; truy vấn danh sách không trả toàn bộ dữ liệu PDF; trạng thái đọc và ẩn chat được lưu theo từng người dùng.
- **Chuẩn hóa nền tảng frontend:** TanStack Query quản lý việc đã lưu và thông báo; React Hook Form kết hợp Zod quản lý biểu mẫu xác thực; Tiptap static renderer và bộ làm sạch thay cách chèn HTML trực tiếp; hook in dùng chung thay các đoạn gọi in rời rạc; Radix UI, Sonner và Error Boundary thống nhất phản hồi và phục hồi lỗi.
- **Liên kết yêu cầu với bằng chứng:** Báo cáo công bố cả 41 trường hợp kiểm thử đạt và các phần chưa được đánh giá như kiểm thử trình duyệt đầu-cuối, tích hợp cơ sở dữ liệu, phân quyền đầy đủ, triển khai serverless và chất lượng ngữ nghĩa AI. Cách trình bày này giúp giới hạn kết luận đúng với bằng chứng hiện có.

8.3 Bài học kinh nghiệm

- Dữ liệu nghề nghiệp cần tách nội dung cấu trúc khỏi bản trình bày. Nếu chỉ lưu PDF, việc đổi template, review theo từng mục và hỗ trợ AI sẽ khó kiểm soát.
- Xóa hoặc ẩn dữ liệu trong hệ thống nhiều bên phải xét quyền sở hữu và nhu cầu lưu lịch sử. Snapshot và trạng thái ẩn theo user phù hợp hơn xóa dây chuyền.
- Kiểm tra ở frontend không thay thế kiểm tra server. Role, userId, MIME hoặc trạng thái Job từ trình duyệt đều phải được xem là dữ liệu không đáng tin.
- Structured output làm giảm lỗi tích hợp AI nhưng không bảo đảm nội dung đúng. Chất lượng mô hình cần rubric, bộ dữ liệu cố định và người chấm độc lập.
- Một PDF “trông đúng” trên màn hình chưa đủ; cần kiểm tra kích thước vật lý, ngắt trang, Print Preview và trình xem nhiều trang.

8.4 Hạn chế hiện tại

Hệ thống chưa được xem là sẵn sàng production. Các hạn chế chính gồm:

- **Lưu trữ:** PDF base64 và ảnh chat dạng data URL làm tài liệu MongoDB lớn.
- **Kiến trúc backend:** nhiều route còn tập trung; thông báo dùng cơ chế lấy định kỳ; Socket.IO trên môi trường serverless chưa được đánh giá độ bền; lịch Vercel Cron chưa có kiểm thử gọi đồng thời hoặc cơ chế quan sát thất bại.
- **Phân quyền:** kiểm tra vai trò còn thiếu ở một số API AI, ứng tuyển, lưu việc và quyền phòng chat.
- **Cấu hình production:** khóa JWT dự phòng, tài khoản quản trị được tạo sẵn và giới hạn ảnh chưa phù hợp môi trường thực tế.
- **Kiểm thử:** bộ kiểm thử chưa chạy với MongoDB thật, trình duyệt đầu-cuối, tải, kiểm thử xâm nhập hoặc mạng không ổn định.
- **Khả năng tái lập và đánh giá AI:** working tree tại thời điểm đánh giá chưa được đóng thành commit hoặc tag tái lập; hiệu quả ngữ nghĩa và tính công bằng của AI chưa được đánh giá độc lập.

8.5 Lộ trình phát triển

Giai đoạn	Công việc ưu tiên
Ngắn hạn	Commit/tag baseline DA2; bắt buộc JWT secret và bỏ tài khoản seed khỏi production; bổ sung kiểm tra vai trò và thành viên phòng; giới hạn ảnh chat; thêm integration test cho Application unique/snapshot, cron và ma trận quyền; chạy checklist trình duyệt.
Trung hạn	Chuyển PDF/ảnh sang object storage hoặc GridFS; tách backend theo miền controller–service–repository; dùng hạ tầng thời gian thực phù hợp serverless; thêm queue/worker, retry và tính lũy đẳng cho tác vụ nền; bổ sung CI, log và audit.
Dài hạn	Xây dựng bộ dữ liệu chấm AI có người đánh giá; theo dõi chi phí/chất lượng theo model; kiểm thử khả năng truy cập, tải và bảo mật; thiết kế quy trình sao lưu, khôi phục và quan sát production.

Bảng 8.2: Lộ trình phát triển đề xuất

8.6 Tổng kết

RabbitCV chứng minh tính khả thi của một ứng dụng web hỗ trợ viết CV có tích hợp AI, đồng thời cho thấy AI chỉ là một thành phần trong hệ thống dữ liệu, quyền truy cập và nghiệp vụ rộng hơn. Working tree ngày 09/08/2026 đáp ứng mục tiêu đề án ở mức nguyên mẫu có thể kiểm chứng: chức năng cốt lõi đã hiện thực, nền tảng giao diện và vận hành được củng cố, 41/41 test đạt, ESLint sạch và build thành công. Những giới hạn về phân quyền, lưu tệp, hạ tầng serverless, khả năng tái lập và đánh giá AI đã được chỉ rõ thành công việc cụ thể thay vì được trình bày như đã hoàn tất.

Tài liệu tham khảo

- [1] Automattic. Mongoose Documentation. <https://mongoosejs.com/docs/>, 2026. Truy cập tháng 08/2026.
- [2] Groq, Inc. Structured Outputs. <https://console.groq.com/docs/structured-outputs>, 2026. Truy cập tháng 08/2026.
- [3] JSON Schema Organization. Understanding JSON Schema. <https://json-schema.org/understanding-json-schema/basics>, 2026. Truy cập tháng 08/2026.
- [4] MongoDB, Inc. MongoDB Manual: Data Modeling. <https://www.mongodb.com/docs/manual/data-modeling/>, 2026. Truy cập tháng 08/2026.
- [5] Mozilla. PDF.js Documentation. <https://mozilla.github.io/pdf.js/>, 2026. Truy cập tháng 08/2026.
- [6] OpenJS Foundation. Express Documentation. <https://expressjs.com/>, 2026. Truy cập tháng 08/2026.
- [7] OpenJS Foundation. Node.js Documentation. <https://nodejs.org/docs/latest/api/>, 2026. Truy cập tháng 08/2026.
- [8] React Hook Form. React Hook Form Resolvers. <https://github.com/react-hook-form/resolvers>, 2026. Truy cập tháng 08/2026.
- [9] React Team. React Documentation. <https://react.dev/>, 2026. Truy cập tháng 08/2026.
- [10] Remix Software. React Router Documentation. <https://reactrouter.com/>, 2026. Truy cập tháng 08/2026.
- [11] Socket.IO. Socket.IO Documentation. <https://socket.io/docs/v4/>, 2026. Truy cập tháng 08/2026.
- [12] Elham Tabassi. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023. <https://doi.org/10.6028/NIST.AI.100-1>.
- [13] Tailwind Labs. Tailwind CSS Documentation. <https://tailwindcss.com/docs>, 2026. Truy cập tháng 08/2026.
- [14] TanStack. TanStack Query Documentation: Overview. <https://tanstack.com/query/latest/docs/framework/react/overview>, 2026. Truy cập tháng 08/2026.
- [15] Vite Team. Vite Guide. <https://vite.dev/guide/>, 2026. Truy cập tháng 08/2026.
- [16] Zod. Zod Documentation. <https://zod.dev/>, 2026. Truy cập tháng 08/2026.

Phụ lục A

Thông tin tái lập và danh mục kỹ thuật

A.1 Quy trình tái lập kiểm tra

Các lệnh sau được thực hiện từ thư mục gốc mã nguồn DA2. `npm ci` phù hợp khi cần cài đúng dependency từ lockfile; lần đánh giá ngày 09/08/2026 sử dụng dependency đã cài và chạy trực tiếp `npm run check`. Kết quả ghi nhận là ESLint không có lỗi/cảnh báo, 41/41 test đạt và Vite 7.3.6 build 1.906 module thành công.

```
npm ci
npm run check
```

Script check thực hiện tuần tự ba lệnh:

```
npm run lint
npm test
npm run build
```

Để chạy ứng dụng phát triển, cần MongoDB và file biến môi trường hợp lệ. Mã nguồn cung cấp script chạy riêng frontend/backend và script điều phối:

```
npm run dev
npm start
npm run start:all
```

Trước khi lưu số liệu kiểm thử chính thức, cần ghi lại ngày giờ, hệ điều hành, phiên bản Node/npm, commit/tag, trạng thái working tree và toàn bộ output của lệnh `npm run check`. Không đưa file `.env`, token, URI có credential hoặc dữ liệu CV thật vào phụ lục.

A.2 Biến môi trường

Tên biến	Mục đích
PORT	Cổng lắng nghe của backend.
MONGODB_URI, MONGO_URI	Chuỗi kết nối MongoDB; MONGO_URI được ưu tiên nếu cả hai cùng tồn tại. Giá trị có thể chứa thông tin xác thực và phải được bảo vệ.
MONGO_DB_NAME	Tên cơ sở dữ liệu; mặc định là RabbitCV.
JWT_SECRET	Khóa ký/xác minh JWT; production phải dùng giá trị ngẫu nhiên mạnh.
NODE_ENV	Phân biệt môi trường phát triển và production.
CLIENT_URL	Bổ sung nguồn frontend được CORS cho phép.
VITE_SOCKET_URL	Ghi đè địa chỉ Socket.IO ở frontend khi không dùng địa chỉ mặc định.
AI_ENABLED	Bật hoặc tắt toàn bộ nhóm chức năng AI.
AI_DAILY_LIMIT	Quota AI theo user trong một ngày.
AI_REQUEST_TIMEOUT_MS	Thời gian tối đa chờ provider cho một request.
GROQ_API_KEY	Khóa truy cập Groq; chỉ đặt ở backend/secret store.
GROQ_MODEL	Tên model dùng cho các workflow AI.
CRON_SECRET	Khóa Bearer bảo vệ endpoint Vercel Cron ở production.

Bảng A.1: Danh sách biến môi trường không kèm giá trị bí mật

A.3 Danh mục API theo miền nghiệp vụ

Bảng A.2 và A.3 dùng prefix để tránh lặp đường dẫn. Ký hiệu :id, :jobId và :room là tham số động.

Miền	Prefix/tài nguyên	Phương thức và thao tác chính
Vận hành	/api/health, /api/admin/health/details, /api/cron/job-expiry	GET trạng thái tối giản; admin đọc chẩn đoán; Vercel Cron gọi kiểm tra hạn tin bằng Bearer secret.
Xác thực	/api/register, /api/login, /api/logout, /api/me	POST đăng ký, đăng nhập, đăng xuất; GET phiên hiện tại.
Hồ sơ User	/api/user, /api/users	GET tìm kiếm/xem profile; PUT profile, security và password.
Resume	/api/resume, /api/resumes	GET danh sách/chi tiết/view; POST tạo hoặc cập nhật; DELETE theo id.
Job	/api/jobs	GET danh sách, chi tiết và Job doanh nghiệp; POST tạo; PUT sửa; DELETE theo id.
Application	/api/jobs/apply, /api/jobs/applications	POST nộp Resume/file; GET hồ sơ; PUT trạng thái Application.
Saved jobs	/api/saved-jobs	GET danh sách; POST lưu; DELETE bỏ lưu theo jobId.

Bảng A.2: Danh mục API theo miền nghiệp vụ (phần 1)

Miền	Prefix/tài nguyên	Phương thức và thao tác chính
Message/chat	/api/messages, /api/chat	GET/POST message; GET user/thread/unread; PATCH đọc hoặc ẩn room.
Notification	/api/notifications	GET danh sách/số chưa đọc; PUT đọc từng mục/tất cả; DELETE theo id.
AI	/api/ai	GET trạng thái/model/quota; POST viết tiểu sử, review, điền CV, chấm CV-Job và từng lượt phỏng vấn.
Admin Job	/api/admin/jobs	GET danh sách; PUT hide/unhide; DELETE theo id.
Admin Company	/api/admin/companies	GET danh sách; PUT trạng thái verification.
Admin User	/api/admin/users	PUT ban; DELETE theo id.
Admin thống kê	/api/admin/stats	GET số liệu dashboard.

Bảng A.3: Danh mục API theo miền nghiệp vụ (phần 2)

A.4 Bất biến dữ liệu cần bảo vệ

Bất biến	Ý nghĩa kiểm thử
Application duy nhất theo Job-candidate	Hai request gần nhau vẫn không tạo hai hồ sơ cho cùng ứng viên và Job.
Application có jobSnapshot	Lịch sử còn tiêu đề, công ty và ngữ cảnh cần thiết sau khi Job bị xóa.
Resume ảo không thuộc thư viện thường	PDF nộp cho một Application không làm rác danh sách CV quản lý của ứng viên.
Danh sách Resume không trả pdfData	Truy vấn metadata không kéo theo toàn bộ base64 và không lộ file ngoài nhu cầu.
Trạng thái chat gắn với user-room	Đánh dấu đọc hoặc ẩn của một phía không thay đổi dữ liệu cá nhân của phía còn lại.
AiUsage không cần nguyên prompt	Quota và quan sát kỹ thuật dùng metadata/hash; dữ liệu CV đầy đủ không được lưu lại chỉ vì mục đích thống kê.
Rich text CV qua danh sách cho phép	Thẻ định dạng được giữ nhưng thuộc tính, script, iframe, object và nội dung nhúng thực thi phải bị loại ở server.
Cron không gửi thông báo lặp theo trạng thái	Các mốc đã thông báo chỉ được cập nhật sau khi tạo notification thành công; cần test thêm gọi đồng thời.

Bảng A.4: Các bất biến quan trọng cho integration test

A.5 Rubric đề xuất cho chất lượng AI

Rubric này dành cho lần đánh giá tiếp theo, không phải số liệu đã thu được trong báo cáo. Mỗi output được chấm theo thang 1–5 rồi quy đổi theo trọng số.

Tiêu chí	Trọng số	Câu hỏi chấm
Bám sát dữ liệu nguồn	25%	Output có giữ đúng sự kiện trong CV/Job và tránh tự thêm tên, thời gian, bằng cấp hay thành tích không có nguồn không?
Đúng dẫn và nhất quán	20%	Nhận xét có mâu thuẫn với dữ liệu, với chính nó hoặc với ràng buộc workflow không?
Khả năng hành động	20%	Người dùng có thể sửa CV hoặc luyện phỏng vấn dựa trên đề xuất cụ thể không?
Rõ ràng và phù hợp	15%	Ngôn ngữ có dễ hiểu, đúng vai trò, đúng độ dài và đúng ngữ cảnh nghề nghiệp không?
An toàn, riêng tư, công bằng	15%	Output có tiết lộ dữ liệu không cần thiết, suy đoán thuộc tính nhạy cảm hoặc tạo lời khuyên phân biệt đối xử không?
Tuân thủ định dạng	5%	JSON/schema, trường bắt buộc, giới hạn danh sách và kiểu dữ liệu có đúng không?

Bảng A.5: Rubric 100 điểm đề xuất cho output AI

Một quy trình tối thiểu nên dùng ít nhất 20 CV đã ẩn danh, trải đều sinh viên, junior và mid-level; ít nhất 10 Job description; hai người chấm độc lập; cùng một model, prompt và tham số. Ngoài điểm trung bình, cần báo cáo tỷ lệ output bị loại vì bịa đặt/vi phạm an toàn và mức đồng thuận giữa hai người chấm. Mẫu nào chứa dữ liệu cá nhân thật chỉ được sử dụng khi có quyền hợp lệ và kế hoạch xóa sau đánh giá.

A.6 Checklist trước khi nộp phiên bản chính thức

- Commit toàn bộ mã nguồn DA2 dùng làm bằng chứng, loại tệp tạm và gắn tag phiên bản.
- Chạy lại `npm run check`; lưu output và cập nhật số liệu nếu thay đổi.
- Chạy checklist thủ công ở Bảng 7.6 và lưu ảnh/kết quả thực tế.
- Xác minh tên đề tài, sinh viên, giảng viên, loại báo cáo và tháng nộp trên bìa.
- Biên dịch LaTeX từ đầu, kiểm tra citation/reference và xem trực quan toàn bộ PDF.
- Bắt buộc cấu hình JWT secret ở production; vô hiệu hóa việc tự tạo tài khoản quản trị trình diễn và không công bố mật khẩu trong Git/PDF.
- Kiểm thử quyền vai trò ở API AI, lưu việc, ứng tuyển; kiểm tra thành viên phòng và giới hạn ảnh chat ở server.
- Loại secret, dữ liệu CV thật, cookie, URI có thông tin xác thực và file môi trường khỏi Git/PDF.